

# A Verifiable Federated Learning Scheme With Privacy-Preserving in MCS

Ke Gu<sup>1b</sup>, Member, IEEE, Jiaqi Lei<sup>1b</sup>, Jingjing Tan<sup>1b</sup>, Member, IEEE, and Xiong Li<sup>1b</sup>, Senior Member, IEEE

**Abstract**—The popularity of edge smart devices and the explosive growth of generated data have driven the development of mobile crowd sensing (MCS). Also, federated learning (FL), as a new paradigm of privacy-preserving distributed machine learning, integrates with MCS to offer a novel approach for processing large-scale edge device data. However, it also brings about many security risks. In this paper, we propose a verifiable federated learning scheme with privacy-preserving for mobile crowd sensing. In our federated learning scheme, the double-layer random mask partition method combined with homomorphic encryption is constructed to protect the local gradients and enhance system security (strong anti-collusion ability) based on the multi-cluster structure of federated learning. Also, a sampling verification mechanism is proposed to allow the mobile sensing clients to quickly and efficiently verify the correctness of their received gradient aggregation results. Further, a dropout handling mechanism is constructed to improve the robustness of mobile crowd sensing-based federated learning. Related experimental results demonstrate that our verifiable federated learning scheme is effective and efficient in mobile crowd sensing environments.

**Index Terms**—Mobile crowd sensing, verifiable federated learning, privacy-preserving, sampling verification.

## I. INTRODUCTION

### A. Background

IN RECENT years, the development of Internet of Things (IoTs) has significantly boosted the popularity of smart devices (such as various sensors), which leads to an explosive growth of generated data [1]. With the widespread deployment of numerous smart devices, the processing and analysis of massive data have become critical. As a key component of the IoT ecosystem, mobile crowd sensing (MCS) leverages ubiquitous mobile sensing devices with user engagement and related technologies to enable large-scale real-time data collection and processing [2], which can further provide

intelligent decision-making and analysis on the sensed data by some machine learning technologies. However, with the increasing demand of privacy-preserving [3], large-scale centralized data collection is becoming almost impossible. This greatly reduces the utilization efficiency of distributed data and leads to data silos. Then, federated learning (FL) is a new paradigm of privacy-preserving distributed machine learning, its combination with MCS can provide a novel approach for intelligently processing large-scale edge device data. The combination of MCS and FL has huge significant advantages, which can effectively process and analyze widely distributed isomorphic or heterogeneous data. This not only optimizes data management and accurate analysis but also is used for more fields, such as traffic flow prediction, urban planning, environmental sensing and monitoring and other fields [4], [5].

As shown in Fig. 1, federated learning, as an innovative distributed machine learning paradigm, enables training participants to collaborate across devices and regions for model training [6]. So, integrating FL into MCS systems [7] can significantly enhance privacy-preserving and data security, particularly in some scenarios involving sensitive information. Moreover, the combination of FL and MCS addresses the issue of data silos. However, this integration also presents some new challenges. On one hand, the current FL framework still risks the privacy of compromised users. Malicious attackers can recover participants' original private data from the local or global gradients in plaintext through monitoring, theft, or collusion, which leads to personal privacy breaches [8], [9], [10]. On the other hand, although mobile crowd sensing can provide the foundational conditions for deploying the FL framework, it faces many issues, such as untrusted data transmissions and heterogeneous computing resources and data. These both require to design a robust and secure FL framework. First, the central party (device) of model training must ensure that the correctness of aggregation results is unaffected by some offline participants. Second, it is a significant security issue for personal training devices to verify the validness of global aggregation model (gradient) returned by the central party in some untrusted environments [11]. Therefore, federated learning in the MCS context faces the challenge of minimizing the negative influences of participants' dropout issues and collusion attacks, whose purpose is to protect training participants' privacies and ensure (or verify) the validness of aggregated models so as to provide high-quality and accurate models.

Then many researchers have proposed some related schemes to solve the above mentioned issues in recent years. In

Received 25 March 2025; revised 31 August 2025; accepted 26 October 2025. Date of publication 31 October 2025; date of current version 29 December 2025. This work was supported by the National Natural Science Foundation of China (No. 62572077, No. 62332018), Open Research Fund of Hunan Provincial Key Laboratory of Intelligent Processing of Big Data on Transportation (No. B202403), Hunan Provincial Social Science Achievement Evaluation Committee (No. XSP25YBC686). The associate editor coordinating the review of this article and approving it for publication was G. Xu. (Corresponding authors: Jingjing Tan; Xiong Li.)

Ke Gu, Jiaqi Lei, and Jingjing Tan are with the School of Computer Science and Technology, Changsha University of Science and Technology, Changsha 410114, China (e-mail: tanjingjing@csust.edu.cn).

Xiong Li is with the Institute for Cyber Security, School of Computer Science and Engineering, University of Electronic Science and Technology of China, Chengdu 611731, China (e-mail: lixiongzhq@163.com).

Digital Object Identifier 10.1109/TNSM.2025.3627581

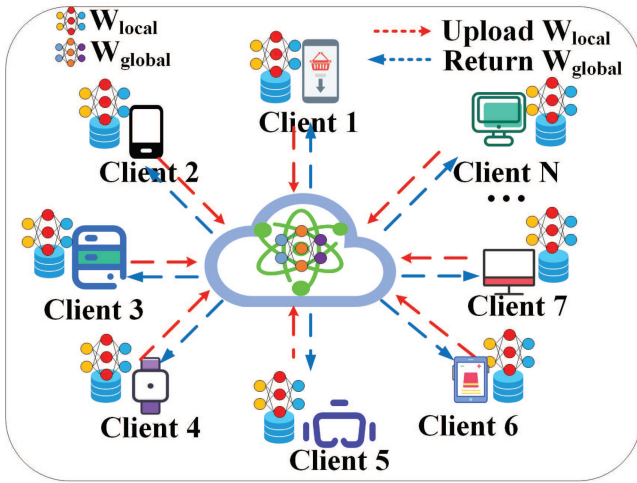


Fig. 1. Traditional federated learning framework.

order to avoid the model in plaintext state being controlled by an adversary, the threshold Paillier encryption method is introduced into the combination of FL and MCS to perform homomorphic encryption on the local model of each participant [12]. However, this method leads to extremely high computational costs for training devices. So, in some practical applications, although this method can protect the privacy of trained model, it cannot resist the privacy leakage issue caused by collusion attacks between the decryption party and other entities. Alternatively, the aggregator oblivious encryption method can be used to encrypt the users' local gradients [13]. However, in this method, the central server decrypts the global gradient locally, which can easily lead to the global gradient being controlled by a malicious central server or stolen during transmission. To avoid the offline training devices influencing the correctness of aggregated results, the secret sharing method is introduced into the FL to deal with the situation where some training devices go offline unexpectedly [14]. However, for the recovery of large-scale offline, this secret sharing method may require high computational resources and communication overhead. Therefore, in this paper we mainly focus on constructing an efficient verifiable federated learning scheme for mobile crowd sensing, which can provide the functions of privacy-preserving, anti-collusion and aggregation result verification even in the case of training device dropouts.

### B. Our Contributions

In this paper, we propose a verifiable and privacy-preserving federated learning scheme tailored for mobile crowd sensing. Unlike existing approaches, our method introduces several key innovations. First, we design a double-layer random mask partitioning technique based on a multi-cluster architecture. When combined with homomorphic encryption, this approach effectively protects local gradients and significantly strengthens system security, particularly by providing strong resistance to collusion. Second, we introduce a sampling-based verification mechanism that enables mobile sensing clients to verify the correctness of the received aggregated

gradients, ensuring the integrity and trustworthiness of the learning process. Furthermore, to address client dropout and communication instability in mobile scenarios, we incorporate a robust dropout-handling mechanism together with the sampling verification strategy, which collectively enhance the robustness and efficiency of federated learning in mobile crowd sensing environments. Our main contributions are summarized as follows:

- 1) We construct a multi-cluster structure of mobile crowd sensing for federated learning, where an improved density peak clustering algorithm is used to construct this multi-cluster structure. Based on this multi-cluster structure, we propose the double-layer random mask partition method. This method, combined with homomorphic encryption, can effectively protect local gradients and enhance system security (strong anti-collusion ability).
- 2) We propose a sampling verification mechanism, where the mobile sensing clients are sampled to verify the correctness of their received gradient aggregation results. In this mechanism, a certain number of the mobile sensing clients from each cluster in each epoch are extracted to verify their received gradient aggregation results, whose purpose is effective and efficient to verify whether these received gradient aggregation results have been tampered with.
- 3) We evaluate and analyze the security and performance of our verifiable federated learning scheme. We test the effectiveness and efficiency of our scheme according to model accuracy and computational overhead in the experiment. Also, we compare our scheme with other existing federated learning schemes (SCAFFOLD [15], FedProx [16], FedAvg [17], and PPverifier [18]). Related experimental results demonstrate that our scheme is effective and efficient in mobile crowd sensing environments.

### C. Organization

The rest of this paper is organized as follows. In Section II, the related works about gradient privacy-preserving and aggregation result verification in FL are discussed. In Section III, the related preliminary knowledge is described. In Section IV, a verifiable federated learning framework in mobile crowd sensing environment is proposed with its corresponding threat model and design objectives. In Section V, a verifiable federated learning scheme with privacy-preserving for mobile crowd sensing is proposed. In Section VI, the correctness and security of the proposed scheme are analyzed. In Section VII, some experiments are made to test the performance of the proposed scheme. Finally, some conclusions are drawn in Section VIII.

## II. RELATED WORK

In recent years, the security of federated learning is concerned by more and more scholars. The main concerns include data leakage [10], inference attack [8], model poisoning attack [19], gradient (or model) privacy-preserving [20] and secure aggregation verification [21], where we focus on the

related works of gradient privacy-preserving and aggregation result verification with MCS in this section.

#### A. Gradient or Model Privacy-Preserving

If the local gradients or model of participated clients and the trained global gradient or model are not protected, then some attackers can easily infer the participants' sensitive privacy data. So, to solve this issue, many gradient or model privacy-preserving schemes have been proposed by some scholars in federated learning. Zhang et al. [20] proposed a privacy-enhancing aggregation federated learning framework for mobile crowd sensing. In their scheme, they developed a secure data aggregation method resistant to collusive attacks by integrating the homomorphic encryption with the hashed Diffie-Hellman key exchange protocol. Lin and Zhang [18] proposed a privacy-preserving and verifiable federated learning method in cloud-edge collaborative computing environments. Their scheme can protect all gradients by the Paillier homomorphic encryption and random number generation technique. Besides, they introduced an additive secret sharing scheme to defend against potential collusive attacks among aggregation server, malicious actors and edge nodes. However, the Paillier homomorphic encryption system in their scheme incurs high computational overhead, leading to low efficiency and scalability of the entire training system.

Additionally, many researchers also adopted some new protection techniques for gradient or model privacy-preserving. Miao et al. [22] designed a blockchain-based privacy-preserving Byzantine robust FL scheme, which can decrease the influence of central server and malicious clients. Hu et al. [23] proposed a blockchain-based mobile crowd sensing learning framework, which incorporates data submission rules to protect the data privacy of mobile devices and ensure the data security of edge servers. Liu et al. [24] introduced a comprehensive FL framework, which employs anonymous masking and certificateless proxy re-encryption to secure data aggregation and local models so as to keep the global model invisible to any parties outside the clients. Their framework also provides the identity authentication function for model uploading. Fu et al. [25] designed a verifiable FL framework for the privacy-preserving of big data in industrial Internet of Things, which applies blinding techniques to protect the privacy of gradients submitted by all participants.

Finally, in order to achieve gradient or model privacy-preserving, many scholars have proposed some security protocols for federated learning. Zhang et al. [26] proposed a lightweight and secure federated learning protocol, which uses the secret sharing method to protect the participant local models uploaded to two servers. Therefore, these two servers do not know the complete participant local training results and cannot infer the participant privacy information. Yuan et al. [27] proposed a privacy-preserving and highly efficient authentication protocol for federated learning in vehicular ad-hoc networks. Their protocol can enhance the vehicular privacy through obfuscating the link between the local model parameters and the identities of corresponding vehicles. Eltaras et al. [19] proposed an efficient and secure

gradient aggregation communication protocol for federated learning. Their protocol uses a single-masking to ensure the confidentiality of local trained gradients. However, their protocol has a security weakness: even if there is not the situation that a client is off-line, the main server can still deceive the auxiliary node into believing there is a single participant dropout, then the auxiliary node returns the off-line user's mask to the main server, which causes the main server to obtain the original gradient related to this user.

#### B. Aggregation Result Verification

Since a central server may be lazy and malicious to tamper with the aggregated result, the trained model may be unusable or have low accuracy. To address this issue, Xu et al. [28] proposed a federated learning framework for privacy-preserving and result verifiability (called as VerifyNet [28]). The core idea of their framework is to use homomorphic hash functions integrated with pseudorandom techniques to verify the correctness of the aggregated result returned by the aggregation server. However, some clients may collude with the aggregation server to forge the aggregated result in their framework. Guo et al. [29] constructed a verifiable aggregation protocol (called as VeriFL [29]), which combines the linear homomorphic hash and the commitment method to force the aggregation server to use the gradients corresponding to the previously broadcast hashing. Their protocol ensures the hash collision resistance and prevents the aggregation server from providing forged aggregation result to any honest clients. However, if an attacker can forge the commitment value and sends the incorrect gradients with hash values to some honest participants, these participants may accept the forged aggregation result.

Further, to address these security issues, Ren et al. [30] analyzed the security of VerifyNet [28] and VeriFL [29], and proposed a new federated learning aggregation protocol. Their protocol combines the linear homomorphic hash and signature technology to enable the traceable verification of aggregated result. Xia et al. [21] proposed a new federated learning scheme for securely verifiable chain aggregation, which can prevent single-point failures by using the Hadamard product to verify the aggregated result. Zhang et al. [31] proposed a verifiable chain-privacy federated learning scheme. Their scheme decreases the communication overhead through uploading the verification tags to the aggregation server, thereby avoiding the reliance on linear homomorphic hash or secret sharing in the verification process. Du et al. [32] proposed a Schnorr protocol-based security aggregation commitment scheme to verify the integrity of aggregated result. Their scheme provides secure result verification without de-constructing or exposing local model updates, and reduces the client verification costs by collectively verifying local model updates on some clients. Zheng et al. [33] utilized the hardware-assisted trusted execution environments (TEE) to ensure the computational integrity of cloud servers in federated learning. However, the use of TEE requires additional hardware resources, which decreases the scalability of FL. Although TEE can guarantee the correctness of aggregated result, additional measures are

needed to address the potential tampering issue of some untrusted network environments.

In summary, we focus on constructing an efficient verifiable federated learning scheme for mobile crowd sensing in this paper, which can provide the functions of privacy-preserving, anti-collusion and aggregation result verification even in the case of training device dropouts.

### III. PRELIMINARIES

#### A. Federated Learning

In federated learning [34], we define the mathematical function expression of the neural network model used by  $N$  participants as  $f(x, G)$  (when these participants are trained locally), where  $x$  is the input and  $G$  is the model parameter information.  $G$  includes the weight and bias between neurons in each layer of neural network. Also, each participant has its own private data  $D_i$  ( $i = 1, 2, \dots, N$ ), where  $D_i = \{(x_s, y_s) | s = 1, 2, \dots, |D_i|\}$ ;  $x_s$  and  $y_s$  represent the sample inputs and the sample labels, respectively; and  $|D_i|$  represents the size of the data  $D_i$ . Also, the loss function  $L_f(D_i, G)$  for each participant in local model training is defined as follows:

$$L_f(D_i, G) = \frac{1}{|D_i|} \sum_{(x_s, y_s) \in D_i} (y_s - f(x_s, G))^2 \quad (1)$$

Then the training process of federated learning is as follows:

- **Model initialization:** The central party or the model provider (generally acted by the cloud server) locally initializes the global model parameter information  $G$ , learning rate  $\alpha$ , local iteration epochs  $e$  and other parameters, and distributes these parameters to each participant.
- **Model training:** After receiving the initialization parameters from the central party, each participant uses its own local data for model training to obtain the local gradient  $W_i = \nabla L_f(D_i^*, G)$ , where  $\nabla L_f$  is the derivative of the loss function  $L_f$  and  $D_i^*$  represents the random sampling of  $D_i$ . Then the local gradient  $W_i$  of each participant is sent to the central party.
- **Model aggregation:** After receiving the local gradient  $W_i$  of each participant, the central party aggregates all these received gradients and obtains the global gradient  $W = \sum_{i=1}^N W_i$ . Then the center party sends the  $W$  to each participant.
- **Model updating:** After receiving the aggregated global gradient  $W$ , each participant calculates a new epoch of model parameter  $G = G - \alpha \cdot \frac{W}{N}$ , then starts the next epoch of model training until the convergence state of trained model is reached.

#### B. CKKS Homomorphic Encryption

Cheon-Kim-Kim-Song (CKKS) [35] is a homomorphic encryption scheme, which is based on the security assumptions of the Learning With Errors (LWE)/Ring Learning With Errors (RLWE). The CKKS homomorphic encryption scheme mainly includes the following algorithms:  $\mathbf{KeyGen}(1^k) \rightarrow (pk, sk, evk)$ ,  $\mathbf{Ecd}(z, \Delta) \rightarrow m$ ,  $\mathbf{Enc}(m, pk) \rightarrow c_t$ ,  $\mathbf{Dec}(c_t, sk) \rightarrow m$ ,  $\mathbf{Dcd}(m, \Delta) \rightarrow z$ ,  $\mathbf{Add}(c_{t1}, c_{t2}) \rightarrow c'_t$ ,

$\mathbf{Mult}(c_{t1}, c_{t2}, evk) \rightarrow \tilde{c}_t$ .  $\mathbf{KeyGen}(1^k)$  is used to generate the keys used for encryption, where  $k$  is the security parameter;  $pk, sk$  and  $evk$  represents the public key, the private key and the evaluation key respectively.  $\mathbf{Ecd}(z, \Delta)$  is used to encode the original vector, where  $z$  is a vector,  $\Delta$  is the scaling factor and  $m$  is the encoded output.  $\mathbf{Enc}(m, pk)$  is used to encrypt the encoded polynomial, where  $c_t$  is the ciphertext.  $\mathbf{Dec}(c_t, sk)$  is used to decrypt the ciphertext.  $\mathbf{Dcd}(m, \Delta)$  is used to decode the encoded polynomial.  $\mathbf{Add}(c_{t1}, c_{t2})$  is used to add the ciphertexts, where  $c_{t1}$  and  $c_{t2}$  are the ciphertexts and  $c'_t$  is the output ciphertext.  $\mathbf{Mult}(c_{t1}, c_{t2}, evk)$  is used to multiply the ciphertexts, where  $c_t$  is the output ciphertext.

#### C. Homomorphic Hash Function

Homomorphic hash function [36], [37] is the operation that can further calculate the hash values after obtaining the hash values from corresponding plain texts. The homomorphic hash function with its related algorithms is described as follows:

- **KeyGen( $1^k$ ):** this algorithm is to generate some system initialization parameters. First, it selects a random number  $k$  as the initial input, and then outputs the system public parameter  $cp = (\mathbb{G}, q, h)$ , where  $q$  represents the prime order,  $\mathbb{G}$  is the cyclic group of the prime order  $q$  and  $h$  is the generator of the group  $\mathbb{G}$ .
- **Hash( $\delta, W$ ):** this algorithm is to map the plaintext  $W$  in a certain domain to a hash space. It inputs a key  $\delta$  and the plaintext message  $W$  with  $W \in \mathbb{Z}_q$ , and then outputs the hash value of the plaintext message,  $H(W) \leftarrow (h^W)^\delta \in \mathbb{G}$ .
- **Properties:** If there are two homomorphic hash values, where  $h_1 = \mathbf{Hash}(\delta, W_1)$  and  $h_2 = \mathbf{Hash}(\delta, W_2)$ , then there must be  $h_1 \otimes h_2 = \mathbf{Hash}(\delta, W_1 + W_2)$ .<sup>1</sup>

### IV. SYSTEM FRAMEWORK AND THREAT MODEL

In this section, we show a verifiable federated learning framework with privacy-preserving in a mobile crowd sensing environment. Further, we propose its corresponding threat model and the design objective based on our work.

#### A. System Framework

In this section, we propose a verifiable federated learning framework in mobile crowd sensing. As shown in Fig. 2, this proposed framework includes three entities: trusted authority, mobile sensing devices (include mobile sensing clients and mobile sensing nodes) and cloud server. The detailed functions of these entities are described as follows:

- **Trusted authority (TA):** Trusted authority can organize the cloud server and mobile sensing devices to participate in federated learning. It is responsible for the distribution of system parameters and the construction of the multi-cluster structure, where all participated mobile sensing devices are divided to multiple different clusters by the trusted authority. In addition, it is responsible for distributing the key pair to each registered mobile sensing

<sup>1</sup>In this paper,  $\oplus$  denotes the addition operation and  $\otimes$  denotes the multiplication operation.

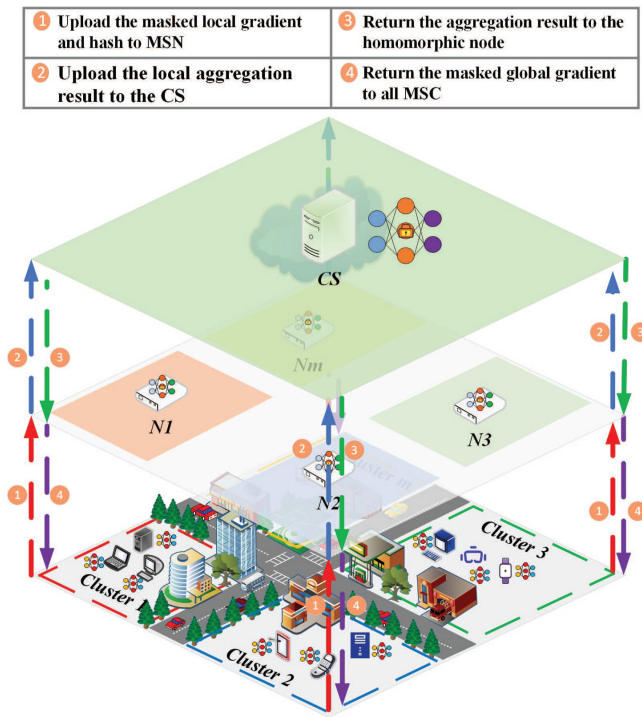


Fig. 2. System framework.

device and auditing the verification results, where if a security issue is found in any epoch, all entities are immediately informed to retrain their model or stop their model training.

- **Cloud server (CS):** Cloud server is responsible for generating the random masks in the initialization phase and partitions them to the participated mobile sensing nodes, and aggregating the ciphertexts of gradients uploaded by the participated mobile sensing nodes.
- **Mobile sensing device (MSD):** Mobile sensing devices are usually some mobile communication devices, workstations and desktop computers, which are further divided to multiple clusters by the trusted authority according to some performance metrics. Also, these mobile sensing devices are categorized as the following two types in each cluster (according to their functions of data processing).
  - 1) **Mobile sensing client (MSC):** Mobile sensing client is responsible for gradient (or local model) training, gradient masking, gradient encryption and gradient hashing. In addition, The mobile sensing clients can verify the correctness of the aggregation result returned by the cloud server and send the verification result to the trusted authority.
  - 2) **Mobile sensing node (MSN):** Mobile sensing node is a mobile sensing device with higher bandwidth and more stable performance than other MSDs in the same cluster, which is responsible for the second partition of random masks and the aggregation of local gradients sent by the MSCs within its cluster.

Based on the functions of these entities, our verifiable federated learning framework mainly consists of three layers: 1) the first layer is the mobile sensing client layer, which is

mainly used for local model training; 2) the second layer is the mobile sensing node layer, which is mainly used to assist in model training, preliminary aggregation and verification; 3) the third layer is the aggregation layer, which is mainly used for final local gradient aggregation by the cloud server.

### B. Threat Model

In this section, we show a threat model for our verifiable federated learning framework. In our threat model, we define that the trusted authority is completely trustworthy and do not collude with any entities. Also, the mobile sensing devices are honest but curious, where we do not consider some internal attacks in this work, such as poisoning attacks. This means that the mobile sensing devices can honestly participate and perform their own tasks in each epoch, but they may be curious about the private information of other entities. Further, the cloud server is semi-honest, which means that it executes the training process but it may only aggregate partial gradients due to its laziness or forge a global gradient without aggregation operations. Therefore, our verifiable federated learning framework system is exposed to the following main security issues:

- **Gradient tampering attacks:** In our federated learning framework, the mobile sensing clients send their local gradients to the corresponding mobile sensing node (which first performs its local aggregation) in each cluster, then the mobile sensing nodes send their local aggregation results to the cloud server, and finally the cloud server performs its global aggregation for its received gradients. Therefore, the mobile sensing clients and the mobile sensing nodes may both tamper with their (received) gradients, and the cloud server may also tamper with the gradients or the global gradient. In addition, the gradients may be tampered with during transmission. Because the mobile sensing devices are honest but curious, we do not consider the tampering behaviors of mobile sensing clients (namely we do not consider the mobile sensing clients are malicious to tamper with their own gradients, such as poisoning attacks). And the mobile sensing nodes may tamper with their received local gradients (from the related MSCs) and the local aggregation result. Further, because the cloud server is semi-honest, it may return a falsely (forged) aggregation result.
- **Collusion attacks:** In our federated learning framework, mobile sensing clients, mobile sensing nodes and cloud server are likely to collude with each other to acquire the gradient information of other mobile sensing clients. When there are some attackers (that may be MSCs, MSNs and CS) in federated learning, they are capable of restoring the original local gradients of non-colluding clients or the original global gradient by exchanging their obtained information.

### C. Design Objectives

In this section, we present the design objectives of our verifiable federated learning framework. Our proposed scheme

is inspired by the following challenges in federated learning: 1) how to resist the tampering attacks for the gradients; 2) how to resist the privacy leakage caused by collusion attacks; 3) how to minimize the negative influences of participants' dropout issues. Therefore, our scheme not only needs to realize the verifiability of aggregation result by these participants, but also needs to resist the collusion of multiple entities within the system. Our design objectives are described as follows:

- *Resistance to tampering attacks*: In the whole federated learning process, whether the processed gradients are encrypted or not, the mobile sensing clients and the mobile sensing nodes may both tamper with their (received) gradients, and the cloud server may also tamper with the gradients or the global gradient. Therefore, our design goal is to ensure that each mobile sensing client can verify the validness of the returned aggregation result.
- *Resistance to collusion attacks*: The mobile sensing clients, mobile sensing nodes and cloud server are likely to collude with each other by exchanging their obtained information. Therefore, our design goal is to resist the privacy leakage risk caused by collusion attacks.
- *Dropout robustness*: Since the mobile sensing clients may drop out during training, this may lead to the cloud server not correctly aggregating the global gradient. Therefore, our design goal is that even if there are partial mobile sensing clients dropping out, they can not influence the validness or correctness of the aggregation result.

## V. MOBILE CROWD SENSING-BASED VERIFIABLE FEDERATED LEARNING SCHEME

In this section, we propose a verifiable federated learning scheme with privacy-preserving under the environment of mobile crowd sensing. Our verifiable federated learning scheme consists of six main phases. In the first phase, the multi-cluster structure of federated learning is constructed. In the second phase, the system initialization is set based on the constructed framework. In the third phase, a gradient-based privacy-preserving method is proposed to protect the local gradient of each mobile sensing client. In the fourth phase, a local gradient-based secure aggregation method is proposed to secure the aggregation process of these local gradients from all mobile sensing clients in different scenarios with different aggregation ways. In the fifth phase, a sample verification scheme is proposed to allow the mobile sensing clients to quickly and efficiently verify the correctness of the returned aggregation results. In the sixth phase, a dropout handling mechanism is proposed to improve the robustness of the whole system. The detailed procedure of our verifiable federated learning scheme is described as follows:

### A. Constructing Multi-Cluster Structure of Federated Learning

In this section, we show how to construct a multi-cluster structure for federated learning, where an improved density peak clustering algorithm is used to construct this multi-cluster structure [38]. Based on our verifiable federated

learning framework, we construct a three-layer-based multi-cluster structure. The constructed multi-cluster framework is a logical structure that organizes all the devices involved in the training according to certain rules. We assume that there are  $Z$  mobile sensing devices that are registered for model training, where the trusted authority divides  $Z$  mobile sensing devices to construct  $M$  clusters based on our clustering metrics (each cluster is represented by  $CL_t$  with  $t = 1, 2, \dots, M$ ). Further, the trusted authority selects  $N$  ( $N < Z$ ) mobile sensing devices as mobile sensing clients and  $M$  ( $M < Z$ ) mobile sensing devices as mobile sensing nodes. The specific steps of constructing a multi-cluster structure are described as follows:

- *Step 1*. The trusted authority assigns an identity identifier  $\hat{i}$  ( $\hat{i} = 1, 2, \dots, Z$ ) to each mobile sensing device registered for model training, and obtains the performance information of these registered devices, including the bandwidth value  $B_{\hat{i}}$ , the remaining battery power value  $P_{\hat{i}}$  and the network connection quality value  $Q_{\hat{i}}$  of the  $\hat{i}$ -th mobile sensing device. Further, the performance information ( $B_{\hat{i}}$ ,  $P_{\hat{i}}$  and  $Q_{\hat{i}}$ ) of each registered device is normalized as follows:

$$B\_norm_{\hat{i}} = \frac{B_{\hat{i}} - \min(B)}{\max(B) - \min(B)}, \quad (2)$$

where  $B\_norm_{\hat{i}}$  is a standardized value,  $\min(B)$  is the minimum of device bandwidth among all registered MSDs, and  $\max(B)$  is the maximum of device bandwidth among all registered MSDs;  $\min(P)$  is the minimum of device remaining battery power among all registered MSDs, and  $\max(P)$  is the maximum of device remaining battery power among all registered MSDs;  $\min(Q)$  is the minimum of device network connection quality among all registered MSDs, and  $\max(Q)$  is the maximum of device network connection quality among all registered MSDs. Based on the standardized performance information of each registered device, our clustering metric is defined as follows:

$$n_{\hat{i}} = w_1 \cdot B\_norm_{\hat{i}} + w_2 \cdot P\_norm_{\hat{i}} + w_3 \cdot Q\_norm_{\hat{i}}, \quad (3)$$

where  $w_1, w_2$  and  $w_3$  are the preset weights.

- *Step 2*. The trusted authority calculates the difference  $nd_{\hat{i}, \hat{j}}$  of clustering metric between any two registered MSCs  $\hat{i}$  and  $\hat{j}$  with  $\hat{i}$  and  $\hat{j} = 1, 2, \dots, Z$ , and locally generates a mobile sensing device clustering matrix  $M$  with the size  $Z \times Z$  based on these differences of clustering metric, where each element  $nd_{\hat{i}, \hat{j}} = |n_{\hat{i}} - n_{\hat{j}}|$  in the matrix  $M$ .
- *Step 3*. The trusted authority transforms the two-dimensional matrix  $M$  to the one-dimensional array  $A'$ ,<sup>2</sup> and then calculates the location index  $pos$  used to determine the neighborhood truncation distance parameter  $m_c$  in  $A'$  as follows:

$$pos = \left\lfloor \frac{Z \cdot (Z - 1) \cdot per}{100} \right\rfloor, \quad (4)$$

<sup>2</sup>The matrix  $M$  is flattened in ascending order into a one-dimensional array  $A'$ .

where  $\lfloor \cdot \rfloor$  means the operation of round down and  $per$  is set to 2 in our scheme.<sup>3</sup> Further, the trusted authority sets the  $m_c = A'[pos + Z]$ .

- *Step 4.* The trusted authority calculates the local density  $\rho_i$  of each mobile sensing device as follows:

$$\rho_i = \sum_{\hat{j}=1, \hat{j} \neq \hat{i}}^Z \exp \left( - \left( \frac{nd_{i,\hat{j}}}{m_c} \right)^2 \right). \quad (5)$$

For a mobile sensing device which is seen as a circle center point with a radius of  $m_c$ , there are more mobile sensing devices within this circle, and the local density of this center point is greater.

- *Step 5.* The trusted authority calculates the offset distance  $\delta_i$  for each mobile sensing device, where  $\delta_i = \min_{\hat{j}: \rho_{\hat{j}} > \rho_i} (nd_{i,\hat{j}})$ . For one MSD with the highest local density, the related calculation is  $\delta_i = \max_{\hat{j}} (nd_{i,\hat{j}})$ .
- *Step 6.* The trusted authority estimates the clustering center points, where each MSD with higher  $\delta_i$  and  $\rho_i$  is regarded as the clustering center point. Besides, each MSD with higher  $\delta_i$  and lower  $\rho_i$  is regarded as the noise point (which does not belong to any cluster).
- *Step 7.* The trusted authority assigns other mobile sensing devices (non-clustering center points) into the clusters where these devices are closer to their clustering centers with higher  $\rho_i$ s. In our scheme, the noise points are regarded (the number is  $N_{noise}$ ) as the MSDs that are not selected to participate in this training. Therefore,  $M$  clusters ( $CL_1, CL_2, \dots, CL_M$ ) are constructed. The mobile sensing devices in all these clusters are regarded as the training devices participated in federated learning, where their number is  $N_{train}$  with  $N_{train} = Z - N_{noise}$ .
- *Step 8.* The trusted authority selects one MSD with good performance (high bandwidth and computation capability) as the mobile sensing node in each cluster, and other mobile sensing devices are regarded as the mobile sensing clients, where the number of MSNs is  $M$ . So, the number of MSCs is  $N = Z - N_{noise} - M$ .
- *Step 9.* Based on the constructed multi-cluster structure, the trusted authority reassigns the unique identifiers  $C_i (i = 1, 2, \dots, N)$  and  $N_t (t = 1, 2, \dots, M)$  to each MSC and MSN, where a third-party cloud server is assigned to complete the aggregation task. The trusted authority generates the list  $L$  of MSCs to the cloud server.

## B. System Initialization

In this section, we show how to complete the system initialization based on our multi-cluster structure for federated learning. In our system initialization, the homomorphic public/private key pairs and the homomorphic hash secret key are generated by the trusted authority, the initialized model parameters and random masks are generated by the cloud server, where the double-layer partition of these random masks

is completed by the cloud server and the mobile sensing nodes, as shown in Fig. 3. The details are described as follows:

- *Step 1.* The trusted authority generates the security parameter  $k$  and the homomorphic hash function secret key  $\delta$ , and then generates the homomorphic public/private key pair  $(pk_t, sk_t)$  for each mobile sensing node with  $t = 1, 2, \dots, M$ . Also, the trusted authority builds a  $T$ -epoch sampling verification matrix. In this matrix, each row represents which mobile sensing clients do the verification in a certain period, and each column represents which epochs a mobile sensing client participates in the verification. Further, the trusted authority sends the homomorphic private key  $sk_t$  to each corresponding mobile sensing node, the secret key  $\delta$  and the information in each column of the  $T$ -epoch sampling verification matrix to the mobile sensing clients. Then all the homomorphic public keys  $(pk_1, pk_2, \dots, pk_M)$  are published to the cloud server and the mobile sensing clients.
- *Step 2.* The cloud server initializes the training parameters, including the batch size  $b$ , the total number  $T$  of training epochs, the learning rate  $\alpha$ , the initial global model parameter  $G$  and the local iteration number  $e$  of mobile sensing client. Then, the cloud server sends these initialized parameters to each mobile sensing client.
- *Step 3.* The trusted authority initializes a random selection table of  $T$ -epoch mobile sensing nodes, which is used to assign which mobile sensing nodes are to act as homomorphic or verification auxiliary nodes in each epoch. In each epoch, any two mobile sensing nodes can be selected, one of them is assigned as the homomorphic node and another of them is assigned as the verification auxiliary node (the homomorphic node and verification auxiliary node in each epoch will be dynamically replaced), where the homomorphic node is used for gradient encryption/decryption and the verification auxiliary node is used for aggregating corresponding gradient hash values. Finally, the trusted authority distributes the random selection table of  $T$ -epoch mobile sensing nodes to the cloud server, the MSNs and the MSCs.
- *Step 4.* The cloud server generates the  $T$  random mask  $R_\theta$  ( $\theta = 1, 2, \dots, T$ ), where the dimension of  $R_\theta$  is consistent to the dimension of global gradient. Then, the cloud server splits each  $R_\theta$  into the  $M$  shares, where the constraint condition is  $R_\theta = \sum_{t=1}^M R_{\theta,t}$ ,  $R_{\theta,t} > 0$ ;  $R_{\theta,t}$  represents the  $t$ -th share of the  $\theta$ -th epoch random mask  $R_\theta$ . Further, the cloud server builds a  $T$ -epoch random mask partition matrix based on these shares. In this matrix, each row represents  $M$  shares of the  $R_\theta$  in one epoch and each column represents the  $T$  shares received by one MSN in the  $T$  epochs. Then the information in each column of the  $T$ -epoch random mask partition matrix is securely sent to each corresponding MSN.
- *Step 5.* The  $t$ -th mobile sensing node receives these corresponding share  $R_{\theta,t}$ s of the  $T$  epochs from the cloud server (where  $\theta = 1, 2, \dots, T$ ), and further divides each  $R_{\theta,t}$  into  $N$  sub-shares, where the constraint condition is  $R_{\theta,t} = \sum_{i=1}^N R_{\theta,t,i}$ ;  $R_{\theta,t,i}$  represents the  $i$ -th sub-share

<sup>3</sup>The value of  $per$  is consistent with the DPC algorithm [38], which is empirically set.

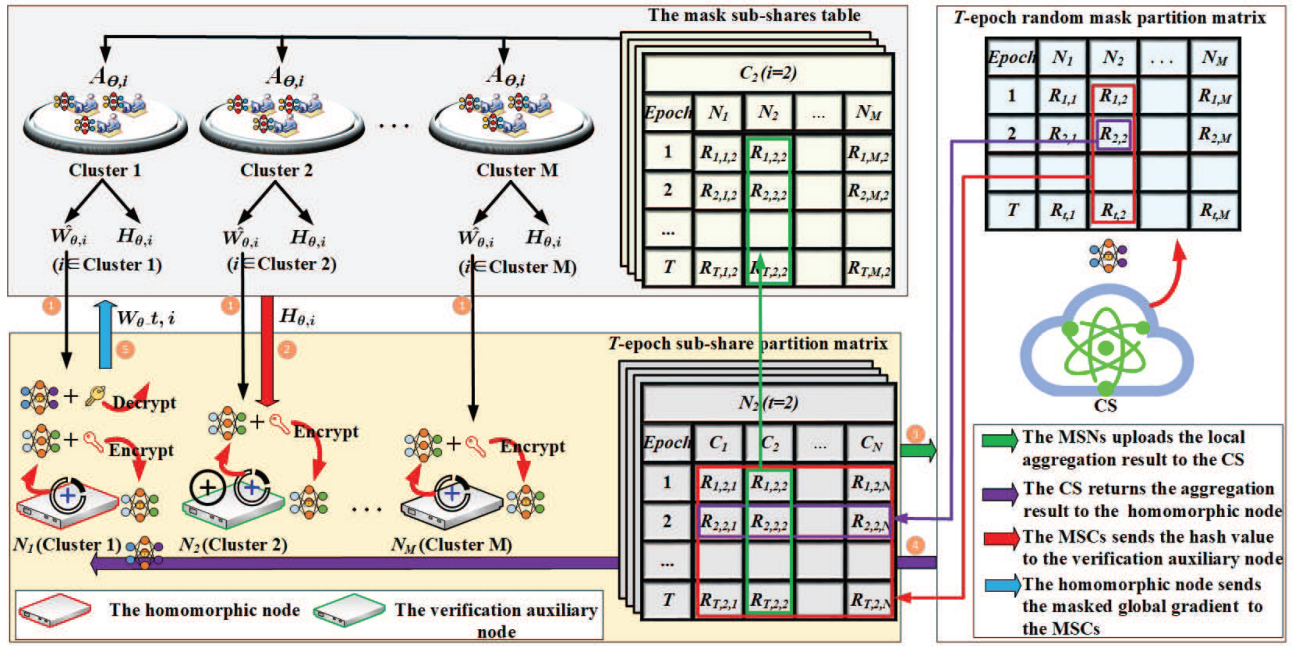


Fig. 3. Random mask double-layer partition.

**Algorithm 1** Local Gradient Training With Privacy-Preserving

**Input:** Original local gradient  $W_{\theta,i}$ , mask sub-share  $R_{\theta,t,i}$  and a key  $\delta$ .

**Output:** Masked gradient  $\hat{W}_{\theta,i}$  and hash value of original gradient  $H_{\theta,i}$ .

- 1: **for** each  $C_i \in \{1, 2, \dots, N\}$  in parallel **do**
- 2:  $C_i$  calculates  $A_{\theta,i} = \sum_{t=1}^M R_{\theta,t,i}$ ;
- 3:  $C_i$  calculates the masked gradient  $\hat{W}_{\theta,i} = W_{\theta,i} + A_{\theta,i}$ ;
- 4:  $C_i$  calculates the homomorphic hash of original local gradient  $H_{\theta,i} \leftarrow \text{Hash}(\delta, W_{\theta,i}) = (h^{W_{\theta,i}})^{\delta}$ ;
- 5: **end for**
- 6: **return**  $\hat{W}_{\theta,i}$  and  $H_{\theta,i}$ ;

of the  $t$ -th share  $R_{\theta,t}$  in epoch  $\theta$ ; the  $R_{\theta,t,i} > 0$ . Similarly, each MSN builds a  $T$ -epoch sub-share partition matrix based on these sub-shares. In this matrix, each row represents  $N$  sub-shares of the  $R_{\theta,t}$  in one epoch and each column represents the  $T$  sub-shares received by one MSN in the  $T$  epochs. Then the information in each column of the  $T$ -epoch sub-share partition matrix is securely sent to each corresponding MSC.

- **Step 6.** Each mobile sensing client locally generates the mask sub-share table according to its received column information of the  $T$ -epoch sub-share partition matrixes from these mobile sensing nodes. In this table, each row represents the  $M$  sub-shares received from the  $M$  MSNs in one epoch and each column represents the  $T$  sub-shares received from one MSN in the  $T$  epochs.

**C. MSC Local Training With Privacy-Preserving**

In this section, we show how to accomplish the local gradient training with privacy-preserving for mobile sensing

clients. In our method, the local gradient and the sum of the received mask sub-shares are calculated and masked by each MSC for each epoch, and then the homomorphic hash value of the original local gradient is calculated by each MSC. The specific steps are described as follows (as shown in Algorithm 1):

- **Step 1.** Each MSC trains its local data to obtain the local gradient  $W_{\theta,i}$ , where  $W_{\theta,i}$  represents the local gradient of the  $i$ -th MSC in the  $\theta$ -th epoch. Then each MSC calculates the sum  $A_{\theta,i}$  of the mask sub-shares in the  $\theta$ -th epoch according to its mask sub-shares table, where  $A_{\theta,i} = \sum_{t=1}^M R_{\theta,t,i}$  and  $A_{\theta,i}$  represents the sum of the sub-shares of the  $i$ -th MSC in the  $\theta$ -th epoch. Finally, the masked gradient is calculated as follows:

$$\hat{W}_{\theta,i} = W_{\theta,i} + A_{\theta,i}, \quad (6)$$

where  $\hat{W}_{\theta,i}$  represents the masked local gradient of the  $i$ -th MSC in the  $\theta$ -th epoch.

- **Step 2.** Each MSC calculates the hash value  $H_{\theta,i}$  of the origin local gradient for the  $i$ -th MSC in the  $\theta$ -th epoch by the homomorphic hash function, where  $H_{\theta,i} \leftarrow \text{Hash}(\delta, W_{\theta,i}) = (h^{W_{\theta,i}})^{\delta}$ .
- **Step 3.** Each MSC sends the  $\hat{W}_{\theta,i}$  to the MSN in its belonged cluster, and the  $H_{\theta,i}$  to the selected verification auxiliary node in this  $\theta$ -th epoch.

**D. Secure Aggregation**

In this section, we show the secure aggregation process among the mobile sensing nodes, the verification auxiliary node and the cloud server. In our scheme, there are three aggregation processes: 1) all these  $\hat{W}_{\theta,i}$ s sent by the MSCs within each cluster are aggregated by the corresponding MSN, and then this local aggregation result is encrypted and sent to the CS by the MSN; 2) all these local aggregation results

uploaded by the MSNs are further aggregated by the CS; 3) all the hashes of the original gradients sent by the MSCs are aggregated by the verification auxiliary node, so as to ensure that the MSCs correctly verify the global aggregation result returned by the CS. The specific steps are described as follows (as shown in Algorithm 2):

- *Step 1.* The MSN in each cluster receives the  $\hat{W}_{\theta,i}$  sent by each MSC in its managing cluster, it aggregates these (received) masked gradients as follows:

$$N_{\theta,t} = \sum_{i \in CL_t} \hat{W}_{\theta,i}, \quad (7)$$

where  $N_{\theta,t}$  represents the local aggregation result with the  $t$ -th cluster in the  $\theta$ -th epoch.<sup>4</sup> Subsequently, each MSN encrypts its local aggregation result  $N_{\theta,t}$  to obtain  $C_{-}N_{\theta,t} \leftarrow \text{Enc}(N_{\theta,t}, pk_t)$ , and generates a gradient upload MSC list  $L_{\theta,t}$ , where  $L_{\theta,t}$  contains the online MSC information within the  $t$ -th cluster in the  $\theta$ -th epoch. Finally, each MSN sends the  $L_{\theta,t}$  and  $C_{-}N_{\theta,t}$  to the CS.

- *Step 2.* After receiving the  $C_{-}N_{\theta,t}$ s and  $L_{\theta,t}$ s from the MSNs, the CS merges these  $L_{\theta,t}$ s to form a total gradient upload MSC list  $L_{\theta}$ , where  $L_{\theta} = \{L_{\theta,1} \cup L_{\theta,2} \cup \dots \cup L_{\theta,M}\}$ . If the CS finds that no MSCs are dropped out by comparing the two lists  $L_{\theta}$  and  $L$ , then it encrypts the  $R_{\theta}$  with the public key of the homomorphic node in this current epoch to obtain  $C_{-}R_{\theta}$ , where  $C_{-}R_{\theta} \leftarrow \text{Enc}(R_{\theta}, pk_t)$  and  $C_{-}R_{\theta}$  represents the ciphertext of  $R_{\theta}$ . Further, the CS aggregates all these received local aggregation results according to the following formula:

$$C_{-}W_{\theta} = \left( \sum_{t=1}^M C_{-}N_{\theta,t} \right) \oplus (-1) \otimes C_{-}R_{\theta}. \quad (8)$$

If the CS checks that there are some MSCs dropped out, then the CS sends the  $L_{\theta}$  to all MSNs. Further, each MSN calculates the sum of the sub-share  $M_{\theta,t}$ s of all online MSCs in this epoch based on the  $L_{\theta}$ , where  $M_{\theta,t} = \sum_{i \in L_{\theta}} R_{\theta,t,i}$ . Finally, each MSN sends its local calculated  $M_{\theta,t}$  to the CS. The CS calculates the sum of these  $M_{\theta,t}$ s uploaded by all the MSNs according to the following formula:

$$M_{\theta} = \sum_{t=1}^M M_{\theta,t}. \quad (9)$$

Also, the CS uses the public key of the homomorphic node in this epoch to encrypt the  $M_{\theta}$  to obtain  $C_{-}M_{\theta} \leftarrow \text{Enc}(M_{\theta}, pk_t)$ , and then the CS aggregates all these local aggregation results according to the following formula:

$$C_{-}W_{\theta} = \left( \sum_{t=1}^M C_{-}N_{\theta,t} \right) \oplus (-1) \otimes C_{-}M_{\theta}. \quad (10)$$

After the above calculations are completed, the CS sends the  $C_{-}W_{\theta}$  to the homomorphic node, where  $C_{-}W_{\theta}$  represents the encrypted global gradient in the  $\theta$ -th epoch.

<sup>4</sup>All these MSCs are divided to form  $M$  clusters, where we use the symbol  $CL_t$  to represent a cluster;  $|CL_t|$  represents the number of MSCs in the  $t$ -th cluster, with  $|CL_1| + |CL_2| + \dots + |CL_M| = N$ .

## Algorithm 2 Secure Aggregation

**Input:** The ciphertext  $C_{-}N_{\theta,t}$ s, the list  $L$  of MSCs, the random mask  $R_{\theta}$ , the sum  $M_{\theta,t}$  of the sub-shares of these online MSCs, the public key  $pk_t$ , and a total gradient upload MSC list  $L_{\theta}$ .

**Output:** The global encrypted gradient  $C_{-}W_{\theta}$

```

1: if  $L == L_{\theta}$  then
2:   CS calculates  $C_{-}R_{\theta} \leftarrow \text{Enc}(R_{\theta}, pk_t)$ ;
3:   CS calculates  $C_{-}W_{\theta} = (\sum_{t=1}^M C_{-}N_{\theta,t}) \oplus (-1) \otimes C_{-}R_{\theta}$ ;
4: else
5:   CS sends the  $L_{\theta}$  to these MSNs;
6:   Each MSN calculates  $M_{\theta,t} = \sum_{i \in L_{\theta}} R_{\theta,t,i}$ ;
7:   These MSNs send the  $M_{\theta,t}$  to the CS;
8:   CS calculates  $M_{\theta} = \sum_{t=1}^M M_{\theta,t}$ ;
9:   CS calculates  $C_{-}M_{\theta} \leftarrow \text{Enc}(M_{\theta}, pk_t)$ ;
10:  CS calculates  $C_{-}W_{\theta} = (\sum_{t=1}^M C_{-}N_{\theta,t}) \oplus (-1) \otimes C_{-}M_{\theta}$ ;
11: end if
12: return  $C_{-}W_{\theta}$ ;

```

- *Step 3.* The verification auxiliary node aggregates the original local gradient hash value  $H_{\theta,i}$ s received from each MSC participating in the training to obtain the global hash value  $H_{\theta} = \prod_{i=1}^N H_{\theta,i}$ . Then, the verification auxiliary node sends the global hash value  $H_{\theta}$  to all these MSCs for final verification.

## E. Sample Verification

In this section, we show the sample verification process. Because the core idea of many traditional federated learning verification schemes is to make each client verify its received aggregation result in each epoch, such method leads to the high cost of system computing resources. Based on our constructed multi-cluster structure, our sample verification process is to extract a certain number of the MSCs from each cluster in each epoch to verify the received global aggregation result, whose purpose is effective and efficient to verify whether the CS has tampered with the result. The specific steps are described as follows:

- *Step 1.* After receiving the  $C_{-}W_{\theta}$  sent by the CS, the homomorphic node uses its own private key  $sk_t$  to decrypt it to obtain the original global gradient  $W_{\theta} \leftarrow \text{Dec}(C_{-}W_{\theta}, sk_t)$ . Then the homomorphic node masks the  $W_{\theta}$  with the corresponding  $R_{\theta,t,i}$  generated in this epoch of initialization (in this phase,  $t$  is the identifier of the homomorphic node in this epoch), which can generate  $N$  different  $W_{\theta-t,i}$ s, and then sends these different  $W_{\theta-t,i}$ s to the corresponding MSCs, where  $W_{\theta-t,i}$  represents the masked global gradient sent by the homomorphic node numbered  $t$  in the epoch  $\theta$  to the  $i$ -th MSC and  $W_{\theta-t,i} = W_{\theta} + R_{\theta,t,i}$ . The purpose of this is to prevent the global gradient from being leaked during transmission.
- *Step 2.* Based on our constructed  $M$  clusters with the  $N$  MSCs, the sampling label  $Tag_{\theta}$  is set to the same

value  $g$  for all MSCs by the TA in the system initialization. Further, we assume  $\hat{c}_{\theta,t}$  mobile sensing clients are extracted from each cluster by the TA in each epoch, where  $\hat{c}_{\theta,t} > 2$ .<sup>5</sup> Therefore, the sample size of each epoch is denoted as  $\hat{C}_\theta (\theta = 1, 2, \dots, T)$ , where the relationship between  $\hat{c}_{\theta,t}$  and  $\hat{C}_\theta$  can be expressed as follows:

$$\hat{C}_\theta = \sum_{t=1}^M \hat{c}_{\theta,t}. \quad (11)$$

Further,  $\hat{C}_\theta$  needs to meet the following conditions:

$$2 \cdot M \leq \hat{C}_\theta \leq N. \quad (12)$$

During the sample verification phase, at least two MSCs are selected from each cluster, and in most cases, all MSCs can be selected to participate in the verification work. Therefore, the total number of MSCs selected must meet condition  $2 \cdot M \leq \hat{C}_\theta \leq N$ , and  $|CL_t|$  needs to satisfy that  $\min\{|CL_t|\} \geq 2$ . Among them, the extracted sample size of each epoch may be inconsistent, and the extracted number of the MSCs in each cluster may also be inconsistent. After the sample extraction process is completed, it is determined that the global aggregation results can be verified by these MSCs.

- *Step 3.* In the  $T$ -epoch sampling verification process, the  $(\theta + 1)$ -th epoch sampling label  $Tag_{-(\theta + 1)}$  is set to  $Tag_{-(\theta + 1)} = Tag_{-\theta} - \vartheta$  by the TA, where  $\vartheta$  is a small constant value. So, in each sampling epoch, the  $Tag_{-\theta}$ s of the extracted MSCs should be reduced by  $\vartheta$ , and the MSCs whose  $Tag_{-\theta}$ s are  $g$  should be extracted first in the next epoch. If there are no MSCs whose  $Tag_{-\theta}$ s are  $g$  in one cluster, then the  $\hat{c}_{\theta,t}$  mobile sensing clients should be extracted according to the values of their  $Tag_{-(\theta + 1)}$ s from large to small.
- *Step 4.* After receiving the corresponding  $W_{\theta-t,i}$  and  $H_\theta$ , the MSCs that have been extracted to participate in the verification calculate the original gradient of  $W_{\theta-t,i}$  according to the locally stored sub-share table to obtain  $W_\theta = W_{\theta-t,i} - R_{\theta,t,i}$ , and then calculate  $H(W_\theta) \leftarrow \text{Hash}(\delta, W_\theta)$ .
- *Step 5.* Each MSC checks whether  $H(W_\theta)$  and  $H_\theta$  are equal, stores the verification result as  $r_i$ , updates the local model and sends the verification result  $r_i$  (whose value is 0 or 1) to the TA, where 1 indicates successful verification and 0 indicates failed verification.
- *Step 6.* TA checks the received  $(r_1, r_2, \dots, r_{\hat{C}_\theta})$ , and forces the CS to stop training if the verification fails on any MSC.

**Theorem 1:** Assuming that the probability that each MSC independently finds the aggregation result to be forged is  $p$ , the probability that at least one MSC in all clusters in each epoch finds the aggregation result to be forged is  $P \geq 1 - e^{-p \cdot \hat{C}_\theta}$ . When the ability of each MSC to independently find that the aggregation result is forged is stronger, namely

the  $p$  approaches 1, then the probability  $P$  that at least one MSC in all clusters finds that the aggregation result is forged approaches  $1 - e^{-\hat{C}_\theta}$  in each epoch, where  $\hat{C}_\theta$  is a number. Further, if  $\hat{C}_\theta$  is bigger (namely the number of extracted MSCs is bigger), then the probability  $P$  approaches 1.

*Proof:* We assume the probability that each MSC independently finds the aggregation result to be forged is  $p$  (the verification outcome of one MSC does not affect that of another, ensuring the independence of detection results among MSCs). So, for the cluster  $CL_t$ , the probability that all extracted MSCs do not find the aggregation result forged is  $P_1 = (1-p)^{\hat{c}_{\theta,t}}$ , then the probability that at least one MSC in the cluster  $CL_t$  finds it forged is  $P_2 = 1 - (1-p)^{\hat{c}_{\theta,t}}$ . Since the verification process within each cluster is independent, the joint probability  $P_3$  that none of these MSCs in all clusters detect a forged aggregation result is the product of these probabilities. So, this probability,  $P_3$ , can be calculated by the following equation:

$$P_3 = \prod_{t=1}^M (1-p)^{\hat{c}_{\theta,t}}. \quad (13)$$

The probability  $P$  that at least one MSC in all clusters finds that the aggregation result is forged is calculated as follows:

$$P = 1 - P_3 = 1 - \prod_{t=1}^M (1-p)^{\hat{c}_{\theta,t}}. \quad (14)$$

Since  $(1-p)^{\hat{c}_{\theta,t}} \leq e^{-p \cdot \hat{c}_{\theta,t}}$ , we can get the following formula:

$$\begin{aligned} P &\geq 1 - \prod_{t=1}^M e^{-p \cdot \hat{c}_{\theta,t}} \\ &\geq 1 - e^{-\sum_{t=1}^M p \cdot \hat{c}_{\theta,t}} \\ &\geq 1 - e^{-p \cdot \sum_{t=1}^M \hat{c}_{\theta,t}} \\ &\geq 1 - e^{-p \cdot \hat{C}_\theta}, \end{aligned} \quad (15)$$

where  $\hat{C}_\theta$  is a number with  $0 < p \leq 1$ . If the ability of each MSC to independently find that the aggregation result is forged is stronger and the number of extracted MSCs is bigger (namely the  $p$  approaches to 1 and  $\hat{C}_\theta \rightarrow \infty$ ), then  $\lim(1 - e^{-p \cdot \hat{C}_\theta}) \rightarrow 1$ . So, Theorem 1 follows.

#### F. Handling MSCs Dropout

In this section, we show how to deal with the MSC dropout issue. In our scheme, when local training is completed, each MSC sends  $\hat{W}_{\theta,i}$  to the MSN within its cluster, and a gradient upload MSCs list  $L_{\theta,t}$  is created by the MSN. Then, the MSN aggregates  $\hat{W}_{\theta,i}$  from all these MSCs in the cluster and transmits  $L_{\theta,t}$  with  $\hat{C}_\theta N_{\theta,t}$  to the CS. So, when receiving the  $L_{\theta,t}$ , the CS locally generates a total gradient upload MSCs online list (denoted as  $L_\theta$ ), then the CS distributes the  $L_\theta$  to each MSN before performing its aggregation. Based on this list, each MSN identifies the online MSCs and excludes the offline MSCs from participating in the calculation of sub-shares. Consequently, each MSN transmits the sum of the online MSCs' sub-shares to the CS. Therefore, this process can ensure that these offline MSCs (absent from the  $L_\theta$ ) do

<sup>5</sup>Its purpose is to avoid a single point of failure in a cluster (where only one MSC is extracted for verification).

not participate in the calculation of sub-shares by any MSNs during training.<sup>6</sup>

## VI. CORRECTNESS AND SECURITY ANALYSIS

In this section, we show a detailed analysis of the correctness and security of our scheme.

### A. Correctness Analysis

In this section, we show the aggregation and verification correctness of our scheme by the following two theorems.

**Theorem 2 (Aggregation Correctness):** If all entities within our federated learning framework honestly perform their operations in our scheme, then all MSCs can obtain the correct global gradient (aggregation result) from the CS.

*Proof:* When all the entities involved in federated learning honestly perform their operations, Equations (8) and (10) can be guaranteed for all these MSCs to obtain the correct global gradient, thus we need to derive and prove the correctness of Equation (10). From the system initialization phase, we can know

$$R_{\theta} = \sum_{t=1}^M R_{\theta,t} = \sum_{t=1}^M \sum_{i=1}^N R_{\theta,t,i}, \quad (16)$$

$$A_{\theta,i} = \sum_{t=1}^M R_{\theta,t,i}. \quad (17)$$

Since we consider that some MSCs may be offline,  $C\_M_{\theta}$  can be rewritten as

$$C\_M_{\theta} = [[M_{\theta}]] = \left[ \left[ \sum_{t=1}^M M_{\theta,t} \right] \right] = \left[ \left[ \sum_{t=1}^M \sum_{i \in L_{\theta}} R_{\theta,t,i} \right] \right]. \quad (18)$$

Therefore, based on the ciphertexts encrypted with the same public key, Equation (10) can be expanded as follows:

$$\begin{aligned} C\_W_{\theta} &= \left( \sum_{t=1}^M C\_N_{\theta,t} \right) \oplus (-1) \otimes C\_M_{\theta} \\ &= \left( \sum_{t=1}^M C\_N_{\theta,t} \right) \oplus (-1) \otimes \left[ \left[ \sum_{t=1}^M M_{\theta,t} \right] \right] \\ &= \left[ \left[ \sum_{i \in CL_1} \hat{W}_{\theta,i} \oplus \cdots \oplus \sum_{i \in CL_M} \hat{W}_{\theta,i} \right] \right] \\ &\quad \oplus (-1) \otimes \left[ \left[ \left( \sum_{i \in L_{\theta}} R_{\theta,1,i} + \cdots + \sum_{i \in L_{\theta}} R_{\theta,M,i} \right) \right] \right] \\ &= \left[ \left[ \sum_{i \in L_{\theta}} (W_{\theta,i} + A_{\theta,i}) \right] \right] \oplus (-1) \otimes \left[ \left[ \sum_{t=1}^M \sum_{i \in L_{\theta}} R_{\theta,t,i} \right] \right] \\ &= \left[ \left[ \sum_{i \in L_{\theta}} W_{\theta,i} + \sum_{i \in L_{\theta}} A_{\theta,i} - \sum_{i \in L_{\theta}} A_{\theta,i} \right] \right] \\ &= \left[ \left[ \sum_{i \in L_{\theta}} W_{\theta,i} \right] \right]. \end{aligned} \quad (19)$$

<sup>6</sup>In our scheme, we consider the MSCs uploading  $\hat{W}_{\theta,i}$  to the MSNs and  $H_{\theta,i}$  to the verification auxiliary node to be one transaction, namely the uploading of these two values is successful or failed simultaneously.

The above proof is based on the presence of the offline MSCs, and further the case without the offline MSCs is also proved by Equation (8). Based on the above process, we can prove that Equation (10) is correct.

**Theorem 3 (Verification Correctness):** Each MSC can independently verify the correctness of the returned global gradient (aggregation result), where our proposed verification mechanism can detect the forged global gradient generated by the CS.

*Proof:* According to our scheme, we can know that each MSC receives a different global gradient in Section V. That is because the homomorphic node adds some corresponding mask sub-shares to the  $N$  original global gradient copies after decrypting the original global gradient. So, the MSCs need to restore the received masked global gradient with their mask sub-shares allocated by the homomorphic node of the current epoch to obtain the original global gradient. Then, each MSC hashes the original global gradient to obtain the hash value and compare it with the hash value received from the verification auxiliary node. If they are equal, the verification is passed. Therefore, we only need to prove the aggregation correctness of the original local gradient hash values (uploaded by the related MSCs) by the verification auxiliary node. We assume that all these MSCs are online, so we know that the aggregation result of the original local gradient hash values by the verification auxiliary node is described as follows:

$$\begin{aligned} H_{\theta} &= \prod_{i=1}^N H_{\theta,i} \\ &= H_{\theta,1} \otimes H_{\theta,2} \otimes \cdots \otimes H_{\theta,N} \\ &= (h^{W_{\theta,1}})^{\delta} \otimes (h^{W_{\theta,2}})^{\delta} \otimes \cdots \otimes (h^{W_{\theta,N}})^{\delta} \\ &= (h^{\sum_{i=1}^N W_{\theta,i}})^{\delta}. \end{aligned} \quad (20)$$

When the global aggregation result is decrypted and masked by the homomorphic node, the different  $W_{\theta,t,i}$ s are sent to all the MSCs. Each MSC computes locally to obtain the global gradient  $W_{\theta}$ , where  $W_{\theta} = W_{\theta,t,i} - R_{\theta,t,i}$ . Further, each MSC computes  $H(W_{\theta}) \leftarrow \text{Hash}(\delta, W_{\theta}) = (h^{W_{\theta}})^{\delta} = (h^{\sum_{i=1}^N W_{\theta,i}})^{\delta}$  locally, then the MSC verifies the correctness of the aggregation result by judging that whether  $H_{\theta}$  and  $H(W_{\theta})$  are equal, where  $H_{\theta}$  is received from the verification auxiliary node. If  $H_{\theta}$  and  $H(W_{\theta})$  are equal, then the verification passes; otherwise, the verification fails.

### B. Security Analysis

In this section, we analyze the security of our scheme by using a simulation-reality paradigm [39], which is proved to provide the privacy security for local and global gradients. Based on our threat model, the privacy security (in the collusion situation) can be assured with at least one honest MSN and two honest MSCs. Additionally, we use the symbol  $U$  to denote the set of MSCs and MSNs, and the cryptographic primitive is parameterized with  $k$ . Also, we use  $U_{\theta}$  to represent the MSC set in which the MSCs successfully upload their encrypted masked gradient and original gradient hash value in the  $(\theta-1)$ -th epoch. We chose the first four epochs to participate

in this demonstration, where  $U_4 \subseteq U_3 \subseteq U_2 \subseteq U_1 \subseteq U$  (which reflects some MSCs may be dropped in different epochs). For a subset  $E \subseteq U \cup CS$ ,  $\mathbf{REAL}_E^{U,k}$  denotes the actual view of these parties in the set  $U$  and  $CS$ , and  $\mathbf{SIM}_E^{U,k}$  represents the simulated view. Further, in the simulation-reality paradigm, the problem to prove the privacy security of local and global gradients is transformed into the problem to be indistinguishable between simulation and reality. So, we construct the simulator  $\mathbf{SIM}$  so that the attacker cannot distinguish between the real protocol execution ( $\mathbf{REAL}$ ) and the “forged data” generated by the simulator ( $\mathbf{SIM}$ ). If the attackers cannot distinguish them, then they cannot extract valid information from the protocol. Further, we demonstrate the privacy security of local and global gradients in our scheme through the following three theorems, where Theorems 4 and 5 are for the privacy of local gradients, and Theorem 6 is for the privacy of global gradient.

**Theorem 4 (Defending Against Collusion Attacks From MSCs and MSNs):** For all  $ks$ ,  $E \subseteq U$ ,  $W_U$ , and  $U_4 \subseteq U_3 \subseteq U_2 \subseteq U_1 \subseteq U$ , there exists a probabilistic polynomial time (PPT) simulator  $\mathbf{SIM}$  whose output is indistinguishable from that of  $\mathbf{REAL}_E^{U,k}$ .

$$\begin{aligned} & \mathbf{REAL}_E^{U,k}(W_U, U_1, U_2, U_3, U_4) \\ & \equiv \mathbf{SIM}_E^{U,k}(W_U, U_1, U_2, U_3, U_4). \end{aligned} \quad (21)$$

Because the simulated view of each entity in  $E$  is indistinguishable from the output of the real view  $\mathbf{REAL}_E^{U,k}$ , even if some MSCs and MSNs collude, then they still could not extract the valid information from our protocol.

*Proof:* The joint view of entities in  $E$  does not depend on the input of entities not in  $E$ . Additionally, the view of the CS is not included in Theorem 4 because we do not consider some attacks from the perspective of the CS, and the MSCs do not share their inputs with all MSNs. To implement the simulation, we have the simulator  $\mathbf{SIM}$  to simulate the real input for the honest but curious MSCs and forged input for the rest MSCs. We consider the simulated view output of MSCs in  $E$  is indistinguishable from the real view output, meaning the output is the same. For the honest MSCs not in  $E$ , the simulator uses some random values instead of true gradients to calculate the masking value  $\hat{W}_{\theta,i}$ . Thus, even without encryption (or if the CS conspires with a homomorphic node), the simulator  $\mathbf{SIM}$  can still generate a masked gradient  $\hat{W}_{\theta,i}$  by using randomly generated indistinguishable numbers. Consequently, even if some MSCs collude with the homomorphic node, the honest but curious MSCs cannot determine whether the aggregated gradient is calculated by using a real gradient or a random number. So, the simulated view of each entity in  $E$  is indistinguishable from the output of the real view  $\mathbf{REAL}_E^{U,k}$ . Even if there is the case that some MSCs and MSNs are colluded, they still could not extract valid information from our protocol. Therefore, Theorem 4 follows.

**Theorem 5 (Defending Against Collusion Attacks From CS, Multi-MSNs and Multi-MSCs):** For all  $ks$ ,  $E \subseteq U \cup CS$ ,  $W_U$ , and  $U_4 \subseteq U_3 \subseteq U_2 \subseteq U_1 \subseteq U$ , there exists a PPT simulator

$\mathbf{SIM}$  whose output is indistinguishable from that of  $\mathbf{REAL}_E^{U,k}$ .

$$\begin{aligned} & \mathbf{REAL}_E^{U,k}(W_U, U_1, U_2, U_3, U_4) \\ & \equiv \mathbf{SIM}_E^{U,k}(W_U, U_1, U_2, U_3, U_4). \end{aligned} \quad (22)$$

The simulator can perfectly simulate the  $\mathbf{REAL}$  without knowing the original gradients of all MSCs, and the output of the simulated view  $\mathbf{SIM}_E^{U,k}$  is indistinguishable from that of the real view  $\mathbf{REAL}_E^{U,k}$ . Therefore, even if some MSCs, some MSNs, and the CS collude, then they still could not extract valid information from our protocol.

*Proof:* To prove the above theorem, we use the standard hybrid argument to prove Theorem 5. The core idea of this proof method is to propose a series of hybrids, with the simulator  $\mathbf{SIM}$  performing step-by-step security modifications according to our proposed scheme. Each hybrid represents a security modification by the simulator. The requirement for each modification is that the previous hybrid and the subsequent hybrid must be indistinguishable. The proof is completed when the simulator can perfectly ensure that the simulated view is indistinguishable from the real view. We use the simulator to simulate a series of random variables, with the specific process shown below:

**Hyb1** In the actual execution of our scheme, the simulator  $\mathbf{SIM}$  first initializes a series of random variables. These random variables represent the joint view of the parties in  $E$  in the actual execution of our scheme.

**Hyb2** In this hybrid, for the honest  $t \in \{U_3 \setminus E\}$  in MSNs, we set  $X = \sum_{t \in \{U_3 \setminus E\}} R_{\theta,t}$ , the simulator uses a random value  $r_{\theta,t}$  to replace  $R_{\theta,t}$  received by these MSNs from the CS, ensuring that the sum of partition results received by these MSNs is equal to  $X$ . This constraint on the generated random number is  $\sum_{t \in \{U_3 \setminus E\}} r_{\theta,t} = \sum_{t \in \{U_3 \setminus E\}} R_{\theta,t}$ . Based on the DDH assumption[40], we consider this hybrid is indistinguishable from **Hyb1**.

**Hyb3** In this hybrid, when the honest  $t \in \{U_3 \setminus E\}$  in MSNs in **Hyb2** divides the mask sub-shares for the randomly generated  $r_{\theta,t}$ , the simulator replaces the value  $R_{\theta,t^*,i}$  in the actual view with a randomly uniform-sized value  $r_{\theta,t^*,i}$ , and the constraint condition of the generated random number is  $\sum_{t \in \{U_3 \setminus E\}} r_{\theta,t,i} = \sum_{t \in \{U_3 \setminus E\}} R_{\theta,t,i}$ . Therefore, for  $i \in \{U_3 \setminus E\}$  in the MSCs, the masked gradients that the simulator generates for them are shown as follows:

$$\hat{W}_{\theta,i} = W_{\theta,i} + \sum_{t \in E} R_{\theta,t,i} + \sum_{t \in \{U_3 \setminus E\}} r_{\theta,t,i}. \quad (23)$$

Therefore, we consider that this hybrid is indistinguishable from **Hyb2**.

**Hyb4** In this hybrid, for each  $i \in \{U_3 \setminus E\}$  in the MSCs, the simulator replaces each original gradient  $W_{\theta,i}$  with a random value  $v_{\theta,i}$ , where the constraint

is  $\sum_{i \in \{U_3 \setminus E\}} v_{\theta,i} = \sum_{i \in \{U_3 \setminus E\}} W_{\theta,i}$ , thus each masked gradient  $\hat{W}_{\theta,i}$  is shown as follows:

$$\hat{W}_{\theta,i} = v_{\theta,i} + \sum_{t \in E} R_{\theta,t,i} + \sum_{t \in \{U_3 \setminus E\}} r_{\theta,t,i}. \quad (24)$$

Then we consider this hybrid is indistinguishable from **Hyb3**.

To sum up, the simulator can perfectly simulate the **REAL** without knowing the original gradients of all MSCs, and the output of the simulated view  $\mathbf{SIM}_E^{U,k}$  is indistinguishable from that of the real view  $\mathbf{REAL}_E^{U,k}$ . Therefore, even if some MSCs, some MSNs, and the CS collude, then they still could not extract valid information from our protocol. Theorem 5 follows.

*Theorem 6 (Privacy Security of Global Gradient):* In our scheme, the CS cannot obtain any valuable information from the aggregation results, namely for the security parameter  $k$  based on the CKKS homomorphic encryption, there is a PPT simulator **SIM** whose output is indistinguishable from that of  $\mathbf{REAL}_{CS}^k$ .

$$\mathbf{REAL}_{CS}^k \equiv \mathbf{SIM}_{CS}^k. \quad (25)$$

So, based on the security of the CKKS homomorphic encryption, the CS only operates on ciphertext and does not obtain any valuable information, where the plaintext-based aggregation result can be obtained only after the global gradient ciphertext is decrypted by the homomorphic node. Therefore, our scheme can realize that the aggregation result is invisible to the CS, which can ensure the privacy security of global gradients to the CS.

*Proof:* Similar to the proof of Theorem 5, we use  $\mathbf{REAL}_{CS}^k$  to represent the real view of the CS, where the simulator **SIM** is used to simulate a series of random variables. The specific proof process is shown below:

**Hyb1** In the actual execution of our scheme, we first initialize a series of random variables with the same distribution as the CS.

**Hyb2** In this hybrid, we use the simulator to modify the operation of each MSC securely. That is, a series of random numbers  $v_{\theta,i}$  with the same local gradient distribution are randomly generated to represent the original gradient  $W_{\theta,i} (i \in U_1)$  of each MSC, so the masked gradient of each MSC is shown below:

$$\hat{W}_{\theta,i} = v_{\theta,i} + \sum_{t=1}^M R_{\theta,t,i}. \quad (26)$$

Each MSC then uploads  $\hat{W}_{\theta,i}$  to each MSN in its cluster.  $\hat{W}_{\theta,i}$  and  $W_{\theta,i}$  are computationally indistinguishable. Therefore, this hybrid is indistinguishable from **Hyb1**.

**Hyb3** In this hybrid, we simulate the security modifications for the operation of MSNs. When each MSN receives the masked gradient  $\hat{W}_{\theta,i} (i \in U_1)$  sent by the MSCs

in the cluster, it computes  $\widetilde{N}_{\theta,t}$  instead of  $N_{\theta,t}$  as follows:

$$\widetilde{N}_{\theta,t} = \sum_{i \in CL_t} \hat{W}_{\theta,i}. \quad (27)$$

Each MSN then encrypts the  $\widetilde{N}_{\theta,t}$  to get the  $\widetilde{C_{-N_{\theta,t}}}$  and sends  $\widetilde{C_{-N_{\theta,t}}}$  to the CS. In the same way, we consider this hybrid is indistinguishable from **Hyb2**.

**Hyb4** In this hybrid, the CS computes  $\widetilde{C_{-W_{\theta}}}$  in the **SIM** according to the following formula:

$$\begin{aligned} \widetilde{C_{-W_{\theta}}} &= \left( \sum_{t=1}^M \widetilde{C_{-N_{\theta,t}}} \right) \oplus (-1) \otimes \gamma \\ &= \left[ \sum_{i \in U_2} \hat{W}_{\theta,i} \right] \oplus (-1) \otimes \gamma, \end{aligned} \quad (28)$$

where

$$\gamma = \begin{cases} C_{-R_{\theta}}, & U_1 = U_2 \\ C_{-M_{\theta}}, & U_1 \neq U_2. \end{cases} \quad (29)$$

In this process, we use  $U_1$  to represent the set of the MSCs identified as participating in training and  $U_2$  to represent the set of the MSCs participating in training in a later epoch. So, if  $U_1 = U_2$ , then no offline MSCs exist; otherwise, there are offline MSCs. Then we can easily see that  $\widetilde{C_{-W_{\theta}}}$  and  $C_{-W_{\theta}}$  are computationally indistinguishable, so this hybrid is indistinguishable from **Hyb3**.

To sum up, we prove that there exists a PPT simulator **SIM** whose output is indistinguishable from the output of  $\mathbf{REAL}_{CS}^k$ . So, based on the security of the CKKS homomorphic encryption, the CS only operates on ciphertext and does not obtain any valuable information, where the plaintext-based aggregation result can be obtained only after the global gradient ciphertext is decrypted by the homomorphic node. Therefore, our scheme can realize that the aggregation result is invisible to the CS, which can ensure the privacy security of global gradients to the CS. Then, Theorem 6 follows.

## VII. EXPERIMENT

In this section, we test and evaluate the performance of our verifiable federated learning scheme through a series of experiments. In our experiments, we use two classical datasets and three neural network models to evaluate the performance metrics of our proposed scheme, such as model accuracy and computational overhead. Further, we compare our scheme with other similar federal learning schemes [15], [16], [17], [18].

### A. Experimental Setting

Our experimental environment is based on a PC running Windows 11, Python 3.10.9 and PyTorch 2.0, configured with an Intel Core i5-12500@3.00 GHz CPU and 32 GB of RAM. We use two datasets, two data segmentation approaches and three neural networks to test our scheme. The detail of our experimental setting is described below:

TABLE I  
NUMBER OF PARAMETERS PER LAYER IN CNN, LENET AND MLP

| Neural network | Layer |       |        |       |      |
|----------------|-------|-------|--------|-------|------|
|                | ConV1 | ConV2 | FC1    | FC2   | FC3  |
| CNN            | 160   | 4640  | 200832 | 1290  | -    |
| LENET          | 156   | 2416  | 30840  | 10164 | 850  |
| MLP            | -     | -     | 15700  | 40200 | 2010 |

1) *Datasets*: We use the MNIST [41] and FashionMNIST [42] datasets for our experiments, which are detailed as follows:

- *MNIST* [41]: The MNIST dataset mainly consists of four parts, training dataset, training dataset label, test dataset and test dataset label. The MNIST dataset contains 70,000 images, including 60,000 images in the training set and 10,000 images in the test set. Each image is composed of 0 to 9 categories of handwritten digits with a pixel size of  $28 \times 28$ .
- *FashionMNIST* [42]: The FashionMNIST dataset is similar to the MNIST dataset in terms of composition, data volume, image size and format and category. Their difference is that the FashionMNIST dataset consists of fashion accessories, such as tops, pants, shoes and handbags.

2) *Data Segmentation Approach*: We simulate two types of data distribution in our experiments, namely independent and identically distributed (IID) and non-independent and identically distributed (Non-IID). The specific data segmentation approaches are described as follows:

- *IID*: For the IID, we randomly shuffle 60,000 images in the training set and then distribute these images equally to each MSC.
- *Non-IID*: For the Non-IID, we sort the data set according to the data label of the training set, and then divide it into 200 groups of 300 data slices, and then assign every two slices to each MSC.

3) *Neural Network*: We test our scheme on three different neural networks (CNN, LENET, and MLP). The specific parameters of each layer of these three neural networks are shown in Table I, where ConV represents the convolutional layer and FC represents the fully connected layer.

4) *Federated Learning Settings*: In our experiment, the number  $N$  of MSCs participating in training is set to 50, the number  $e$  of local iterations is set to 2, the batch size  $b$  is set to 64, the learning rate  $\alpha$  is set to 0.005, the number of communication epochs  $T$  is set to 50, the loss function is set to cross-entropy, and the optimizer is set to Adam.

## B. Experimental Results

In this section, we evaluate and analyze our obtained experimental results, where the model accuracy and the computational overhead are evaluated. Also, we compare our verifiable federated learning scheme with other existing federated learning schemes (SCAFFOLD [15], FedProx [16], FedAvg [17], and PPverifier [18]). The specific experimental results and analysis are shown below.

1) *Model Accuracy*: In this section, the accuracy of our scheme on the MNIST and FashionMNIST datasets under

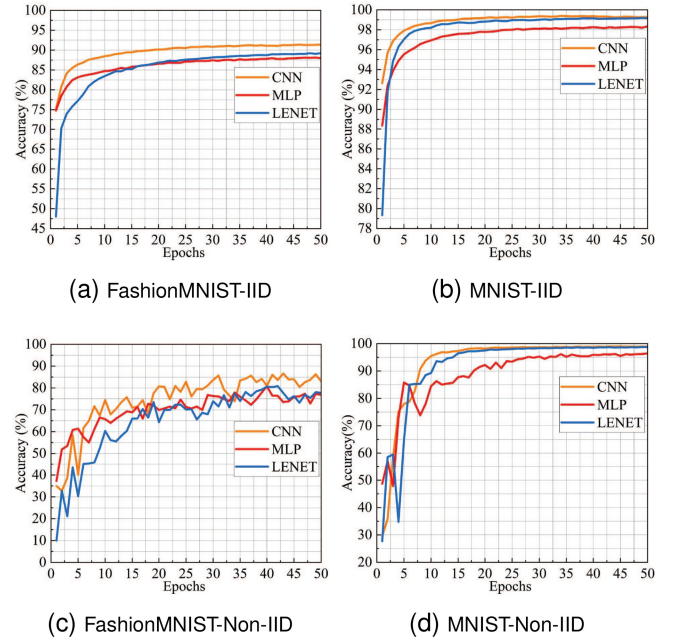


Fig. 4. Accuracy evaluation of three neural networks.

the IID and Non-IID scenarios is evaluated with three neural networks. Further, we also test the accuracy of our scheme for different drop rates on the FashionMNIST under the Non-IID scenario, where the drop rate of MSC is set to 0, 10%, 20%, 30%, and 40%, respectively. Also, the specific parameters of three neural networks are shown in Table I. Fig. 4 shows these related experimental results.

As shown in Fig. 4(a) and Fig. 4(b), when the training epochs increase, the accuracy of our scheme gradually increases on three neural networks. Specifically, on the FashionMNIST dataset, the accuracy of our scheme achieves 91.39% based on CNN, 88.13% based on MLP and 89.22% based on LENET in the 50-th epoch. On the MNIST dataset, the accuracy of our scheme achieves 99.39% based on CNN, 98.34% based on MLP, and 99.19% based on LENET in the 50-th epoch. At the beginning of model training, our scheme has the lowest accuracy on LENET. However, as the training proceeds, the accuracy of our scheme on LENET gradually improves and stabilizes at the later stage, and its overall performance is between that of CNN and MLP. This is because although the LENET has two more convolutional layers than that of the MLP, it has fewer parameters and its initial weight parameters are randomly initialized. So, this initially leads to unstable outputs and less pronounced learning features. As the training continues, the LENET gradually converges, resulting in the improved performance. As shown in Figures 4(c) and 4(d), on the FashionMNIST dataset, the accuracy of our scheme on three neural networks shows an oscillating upward trend with the increase of training epochs (compared to the IID data distribution), and the accuracy of our scheme achieves 86.27% based on CNN, 80.7% based on MLP, and 80.83% based on LENET in the 50-th epoch. On the MNIST dataset, the accuracy of our scheme achieves 99.01% based on CNN, 96.41% based on MLP, and 98.76% based on LENET in the 50-th epoch.

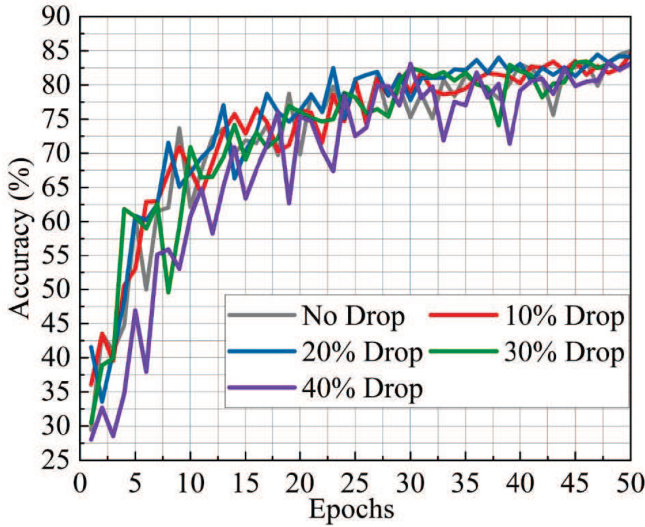


Fig. 5. Accuracy influence of different dropout rates.

Additionally, the model accuracy under the Non-IID scenario shows the oscillatory trend, which is less smooth compared to that of the IID scenario. This is because the significant variations in data distributions across different MSCs lead to these inconsistent (or even conflicting) model update directions. As a result, the global model struggles to converge in a stable direction after each aggregation, causing more pronounced fluctuations during model training. In summary, compared to the other two neural networks, our scheme performs best on CNN under both IID and Non-IID scenarios. Further, based on the superior performance of the CNN model, we test the accuracy of our scheme for different drop rates. As shown in Fig. 5, an increase in the number of MSCs dropping out leads to a decline in the accuracy of our scheme under the same number of training epochs. In federated learning, more sample data from MSCs can facilitate earlier convergence and better accuracy. However, the differences under these dropout rates in our experiment remain relatively small due to the limited number of MSCs and training epochs.

2) *Computational Overhead*: In this section, we first test and evaluate the encryption and decryption costs of two homomorphic encryption algorithms (CKKS and Paillier), where we adopt the CKKS in our scheme due to its higher efficiency. In the experiment, we set the key length of the Paillier scheme [43] to 1024 bits; we referred to relevant study [44], [45] and set  $poly\_modulus\_degree = 16384$ ,  $coeff\_mod\_bit\_sizes = [60, 40, 40, 60]$  and  $global\_scale = 2^{40}$  for the CKKS scheme. This configuration can provide significantly higher security than that of the Paillier scheme with a 1024-bit key. Further, we also test the could server's computational overhead in our scheme (based on CNN) for different drop rates on the FashionMNIST under the Non-IID scenario, where the drop rate of MSC is set to 0, 10%, 20%, 30%, and 40%, respectively. Finally, we test the verification overhead of MSCs based on three neural networks.

As shown in Fig. 6, the encryption and decryption costs of these two algorithms are proportional to the number of model

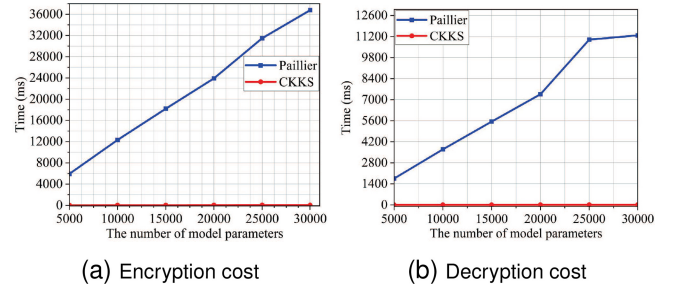


Fig. 6. Encryption and decryption costs under CKKS and Paillier.

TABLE II  
DECRYPTION COST (MS)

| Parameters | 5000    | 10000    | 15000    | 20000    | 25000    | 30000    |
|------------|---------|----------|----------|----------|----------|----------|
| Schemes    |         |          |          |          |          |          |
| CKKS       | 8.52    | 17.04    | 16.67    | 27.93    | 38.57    | 36.06    |
| Paillier   | 5942.23 | 12321.45 | 18205.85 | 23924.91 | 31480.26 | 36780.32 |

TABLE III  
ENCRYPTION COST (MS)

| Parameters | 5000    | 10000   | 15000   | 20000   | 25000    | 30000    |
|------------|---------|---------|---------|---------|----------|----------|
| Schemes    |         |         |         |         |          |          |
| CKKS       | 2.00    | 3.99    | 3.51    | 6.51    | 7.54     | 8.56     |
| Paillier   | 1757.43 | 3696.66 | 5541.69 | 7363.11 | 10993.99 | 11278.73 |

parameters. From Fig. 6, we can know that the encryption and decryption overhead of the Paillier algorithm (blue line) increases gradually as the model parameters increase. Also, the encryption and decryption overhead of the CKKS algorithm (red line) is not clear. This is because the CKKS's encryption and decryption times are very small (at the millisecond level). To show a more intuitive comparison of encryption and decryption costs, we show the cost details in Table II and Table III. The results indicate that Paillier's encryption time increases linearly with the number of model parameters, as it encrypts each element individually. In contrast, the CKKS's encryption time also grows with the number of model parameters but at a significantly slower rate, ranging from 8.5ms to 36ms. So, our CKKS-based verifiable federated learning scheme is efficient.

Additionally, we test the computational overhead of the cloud server based on CNN. As shown in Fig. 7(a), an increase in the number of MSCs dropping out leads to a reduction in overall computational overhead. The black line shows the scenario where no MSCs drop out, resulting in the highest computational overhead across all epochs. This is because a greater number of local gradients received by the CS during aggregation linearly increases its computational overhead. In contrast, when some MSCs drop out, the CS processes fewer local gradients, thereby reducing its overhead. However, Fig. 7(a) also indicates that in certain epochs, the overhead in the dropout scenario may exceed that of the no-dropout scenario. This fluctuation is attributed to background processes or resource contention, which are further amplified during dropout simulations due to additional task scheduling and resource competition.

Also, in the experiments related to verifying the aggregation results, each MSC truncates the floating-point values of gradients before encryption and hashing. This ensures that, in the

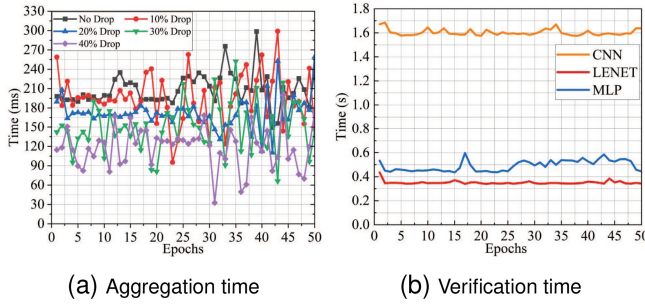


Fig. 7. Computational overhead of cloud server.

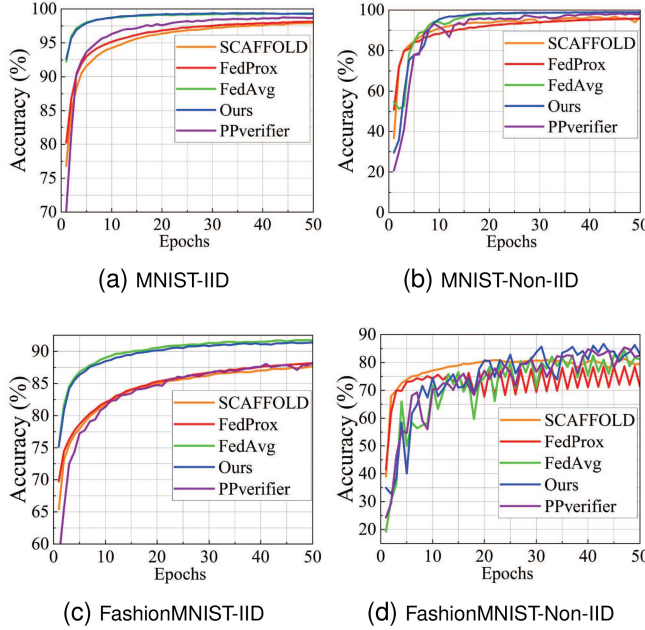


Fig. 8. Accuracy comparison of our scheme with other schemes.

absence of malicious behaviors such as tampering, the hash value of the decrypted global gradient remains consistent with that of the original global gradient. As shown in Fig. 7(b), we can know that our scheme has different verification costs on three neural networks, where the CNN has the longest verification time, followed by the MLP, and the LENET has the shortest. This difference is primarily attributed to the varying number of parameters in the three networks: the CNN has 206,922 parameters, the LENET has 44,426 parameters, and the MLP has 57,910 parameters. So, the time to compute the hash of the global gradient is directly influenced by the number of parameters in the gradient, as each parameter must be hashed. Although the CNN requires a longer verification time, it remains relatively stable at approximately 1.6 seconds, thus this verification time is considered efficient.

3) *Comparison With Other Federated Learning Schemes:* In this section, we compare our verifiable federated learning scheme with other similar schemes SCAFFOLD [15], FedProx [16], FedAvg [17] and PPverifier [18] in terms of model accuracy. These experiments are conducted on these schemes by using different datasets and data segmentation approaches. These comparison results are shown in Figure 8.

As shown in Fig. 8(a) and Fig. 8(b), we evaluate the five schemes on the MNIST dataset by using the IID and Non-IID data segmentation approaches. The experimental results show that in the IID scenario, the accuracy of our scheme is comparable to that of the FedAvg but higher than that of the other three schemes SCAFFOLD, FedProx, and PPverifier, and the model accuracy of our scheme is relatively high in the early stages of model training. In the Non-IID scenario, the overall accuracy of our scheme remains comparable to that of the FedAvg. However, the accuracy difference relative to the other three schemes SCAFFOLD, FedProx, and PPverifier is smaller than that of the IID scenario. As shown in Fig. 8(c) and Fig. 8(d), we test the five schemes on the FashionMNIST dataset by using the IID and Non-IID data segmentation approaches. The results show that the FedAvg achieves the highest accuracy in the IID scenario, followed closely by our scheme, with trends similar to those observed in the MNIST-IID scenario. In the Non-IID scenario, although our scheme performs poorly in the pre-training, our scheme outperforms the other four schemes in general as the model training goes on. Therefore, our verifiable federated learning scheme has some advantages in the whole performance.

## VIII. CONCLUSION

The popularity of edge smart devices and the explosive growth of generated data have driven the development of mobile crowd sensing-based federated learning. However, mobile crowd sensing-based federated learning also brings about many security risks. In this paper, we propose a verifiable federated learning scheme with privacy-preserving for mobile crowd sensing. We first construct a multi-cluster structure of mobile crowd sensing for federated learning. Based on this multi-cluster structure, we propose the double-layer random mask partition method with homomorphic encryption to protect the local gradients in model training. Also, we propose a verifiable federated learning scheme by the sampling verification mechanism, where the mobile sensing clients are sampled to quickly and efficiently verify the correctness of their received gradient aggregation results. Further, we build a dropout handling mechanism to improve the robustness of mobile crowd sensing-based federated learning. Related experimental results show that our verifiable mobile crowd sensing-based federated learning scheme is effective and efficient.

## REFERENCES

- [1] J. Wang, M. K. Lim, C. Wang, and M.-L. Tseng, "The evolution of the Internet of Things (IoT) over the past 20 years," *Comput. Ind. Eng.*, vol. 155, May 2021, Art. no. 107174.
- [2] A. Capponi, C. Fiandrino, B. Kantarci, L. Foschini, D. Kliazovich, and P. Bouvry, "A survey on mobile crowdsensing systems: Challenges, solutions, and opportunities," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 3, pp. 2419–2465, 3rd Quart., 2019.
- [3] O. Reis, N. E. Eneh, B. Ehimuan, A. Anyanwu, T. Olorunsogo, and T. O. Abrahams, "Privacy law challenges in the digital age: A global review of legislation and enforcement," *Int. J. Appl. Res. Social Sci.*, vol. 6, no. 1, pp. 73–88, 2024.
- [4] Z. Jiang et al., "CrowdPatrol: A mobile crowdsensing framework for traffic violation hotspot patrolling," *IEEE Trans. Mobile Comput.*, vol. 22, no. 3, pp. 1401–1416, Mar. 2023.

- [5] J. Liu, H. Shen, H. S. Narman, W. Chung, and Z. Lin, "A survey of mobile crowdsensing techniques: A critical component for the Internet of Things," *ACM Trans. Cyber-Phys. Syst.*, vol. 2, no. 3, pp. 1–26, 2018.
- [6] Y. Jiang, W. Zhang, and Y. Chen, "Data quality detection mechanism against label flipping attacks in federated learning," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 1625–1637, 2023.
- [7] Y. Wang, Z. Su, N. Zhang, and A. Benslimane, "Learning in the air: Secure federated learning for UAV-assisted crowdsensing," *IEEE Trans. Netw. Sci. Eng.*, vol. 8, no. 2, pp. 1055–1069, Apr.–Jun. 2021.
- [8] K. Ganju, Q. Wang, W. Yang, C. A. Gunter, and N. Borisov, "Property inference attacks on fully connected neural networks using permutation invariant representations," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security*, 2018, pp. 619–633.
- [9] N. Carlini, C. Liu, Ú. Erlingsson, J. Kos, and D. Song, "The secret sharer: Evaluating and testing unintended memorization in neural networks," in *Proc. 28th USENIX Security Symp. (USENIX security)*, 2019, pp. 267–284.
- [10] H. Gong et al., "Gradient leakage attacks in federated learning," *Artif. Intell. Rev.*, vol. 56, no. S1, pp. 1337–1374, 2023.
- [11] C. Hahn, H. Kim, M. Kim, and J. Hur, "VerSA: Verifiable secure aggregation for cross-device federated learning," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 1, pp. 36–52, Jan./Feb. 2021.
- [12] B. Zhao, X. Liu, W.-N. Chen, and R. H. Deng, "CrowdFL: Privacy-preserving mobile crowdsensing system via federated learning," *IEEE Trans. Mobile Comput.*, vol. 22, no. 8, pp. 4607–4619, Aug. 2023.
- [13] Y. Wang, A. Zhang, S. Wu, and S. Yu, "VOSA: Verifiable and oblivious secure aggregation for privacy-preserving federated learning," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 5, pp. 3601–3616, Sep./Oct. 2023.
- [14] Z. Yang, M. Zhou, H. Yu, R. O. Sinnott, and H. Liu, "Efficient and secure federated learning with verifiable weighted average aggregation," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 1, pp. 205–222, Feb. 2023.
- [15] S. P. Karimireddy, S. Kale, M. Mohri, S. Reddi, S. Stich, and A. T. Suresh, "SCAFFOLD: Stochastic controlled averaging for federated learning," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 5132–5143.
- [16] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proc. Mach. Learn. Syst.*, vol. 2, 2020, pp. 429–450.
- [17] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. Artif. Intell. Stat.*, 2017, pp. 1273–1282.
- [18] L. Lin and X. Zhang, "PPVerifier: A privacy-preserving and verifiable federated learning method in cloud-edge collaborative computing environment," *IEEE Internet Things J.*, vol. 10, no. 10, pp. 8878–8892, May 2023.
- [19] T. Eltaras, F. Sabry, W. Labda, K. Alzoubi, and Q. Ahmedeltaras, "Efficient verifiable protocol for privacy-preserving aggregation in federated learning," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 2977–2990, 2023.
- [20] M. Zhang, S. Chen, J. Shen, and W. Susilo, "PrivacyEAF: Privacy-enhanced aggregation for federated learning in mobile crowdsensing," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 5804–5816, 2023.
- [21] Y. Xia, Y. Liu, S. Dong, M. Li, and C. Guo, "SVCA: Secure and verifiable chained aggregation for privacy-preserving federated learning," *IEEE Internet Things J.*, vol. 11, no. 10, pp. 18351–18365, 2024.
- [22] Y. Miao, Z. Liu, H. Li, K.-K. R. Choo, and R. H. Deng, "Privacy-preserving Byzantine-robust federated learning via blockchain systems," *IEEE Trans. Inf. Forensics Security*, vol. 17, pp. 2848–2861, 2022.
- [23] Q. Hu, Z. Wang, M. Xu, and X. Cheng, "Blockchain and federated edge learning for privacy-preserving mobile crowdsensing," *IEEE Internet Things J.*, vol. 10, no. 14, pp. 12000–12011, Jul. 2023.
- [24] J. Liu et al., "A comprehensive privacy-preserving federated learning scheme with secure authentication and aggregation for Internet of Medical Things," *IEEE J. Biomed. Health Inform.*, vol. 28, no. 6, pp. 3282–3292, Jun. 2024.
- [25] A. Fu, X. Zhang, N. Xiong, Y. Gao, H. Wang, and J. Zhang, "VFL: A verifiable federated learning with privacy-preserving for big data in industrial IoT," *IEEE Trans. Ind. Informat.*, vol. 18, no. 5, pp. 3316–3326, May 2022.
- [26] Z. Zhang et al., "LSFL: A lightweight and secure federated learning scheme for edge computing," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 365–379, 2022.
- [27] X. Yuan et al., "FedComm: A privacy-enhanced and efficient authentication protocol for federated learning in vehicular ad-hoc networks," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 777–792, 2024.
- [28] G. Xu, H. Li, S. Liu, K. Yang, and X. Lin, "VerifyNet: Secure and verifiable federated learning," *IEEE Trans. Inf. Forensics Security*, vol. 15, pp. 911–926, 2019.
- [29] X. Guo, Z. Liu, J. Li, J. Gao, B. Hou, C. Dong, and T. Baker, "VeriFL: Communication-efficient and fast verifiable aggregation for federated learning," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 1736–1751, 2020.
- [30] Y. Ren, Y. Li, G. Feng, and X. Zhang, "Privacy-enhanced and verification-traceable aggregation for federated learning," *IEEE Internet Things J.*, vol. 9, no. 24, pp. 24933–24948, 2022.
- [31] Z. Zhang et al., "G-VCFL: Grouped verifiable chained privacy-preserving federated learning," *IEEE Trans. Netw. Service Manag.*, vol. 19, no. 4, pp. 4219–4231, Dec. 2022.
- [32] R. Du, X. Li, D. He, and K.-K. R. Choo, "Toward secure and verifiable hybrid federated learning," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 2935–2950, 2024.
- [33] Y. Zheng, S. Lai, Y. Liu, X. Yuan, X. Yi, and C. Wang, "Aggregation service for federated learning: An efficient, secure, and more resilient realization," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 2, pp. 988–1001, Mar./Apr. 2023.
- [34] H. B. McMahan, E. Moore, D. Ramage, and B. Agüera y Arcas, "Federated learning of deep networks using model averaging," 2016, *arXiv:1602.05629*.
- [35] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic encryption for arithmetic of approximate numbers," in *Proc. 23rd Int. Conf. Theory Appl. Cryptol. Inf. Security*, 2017, pp. 409–437.
- [36] W. Xu, D. Feng, and J. Liu, "Remote data integrity checking protocols from homomorphic hash functions," in *Proc. IEEE 14th Int. Conf. Commun. Technol.*, 2012, pp. 604–608.
- [37] H. Yao, C. Wang, B. Hai, and S. Zhu, "Homomorphic hash and blockchain based authentication key exchange protocol for strangers," in *Proc. 6th Int. Conf. Adv. Cloud Big Data (CBD)*, 2018, pp. 243–248.
- [38] A. Rodriguez and A. Laio, "Clustering by fast search and find of density peaks," *Science*, vol. 344, no. 6191, pp. 1492–1496, 2014.
- [39] O. Goldreich, S. Micali, and A. Wigderson, "How to play any mental game, or a completeness theorem for protocols with honest majority," in *Providing Sound Foundations for Cryptography: On the Work of Shafi Goldwasser and Silvio Micali*. New York, NY, USA: ACM, 2019, pp. 307–328.
- [40] D. Boneh and M. Franklin, "Identity-based encryption from the Weil pairing," in *Annu. Int. Cryptol. Conf.*, 2001, pp. 213–229.
- [41] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.
- [42] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms," 2017, *arXiv:1708.07747*.
- [43] D. Catalano, R. Gennaro, N. Howgrave-Graham, and P. Q. Nguyen, "Paillier's cryptosystem revisited," in *Proc. 8th ACM Conf. Comput. Commun. Security*, 2001, pp. 206–214.
- [44] H. Choi, J. Kim, S. Kim, S. Park, J. Park, W. Choi, and H. Kim, "UniHENN: Designing faster and more versatile homomorphic encryption-based CNNs without im2col," *IEEE Access*, vol. 12, pp. 109323–109341, 2024.
- [45] Y. Pan, Z. Chao, W. He, Y. Jing, L. Hongjia, and W. Liming, "FedSHE: privacy preserving and efficient federated learning with adaptive segmented CKKs homomorphic encryption," *Cybersecurity*, vol. 7, no. 1, p. 40, 2024.



**Ke Gu** (Member, IEEE) received the Ph.D. degree from the School of Information Science and Engineering, Central South University, Changsha, China, in 2012. In 2013, he joined the School of Computer Science and Technology, Changsha University of Science and Technology, where he is an Associate Professor. He has published over 100 research papers in journals and conferences. His research interests include artificial intelligence security, IoTs, VANETs, social networks, and network and information security.



**Jiaqi Lei** received the M.S. degree from the School of Computer Science and Technology, Changsha University of Science and Technology, Changsha, China, in 2025. His main interests include artificial intelligence security.



**Xiong Li** (Senior Member, IEEE) received the Ph.D. degree in computer science and technology from the Beijing University of Posts and Telecommunications, Beijing, China, in 2012. He is currently a Full Professor with the School of Computer Science and Engineering, University of Electronic Science and Technology of China, Chengdu, China. He has published over 150 referred papers. His current research interests include artificial intelligence security, cryptography, and information security.



**Jingjing Tan** (Member, IEEE) received the B.E. and M.S. degrees from the School of Computer Science and Technology, Changsha University of Science and Technology, Changsha, China, in 2017 and 2020, respectively, and the Ph.D. degree from the School of Computer Science and Engineering, Central South University, Changsha, in 2024. He is currently an Assistant Professor with the School of Computer Science and Technology, Changsha University of Science and Technology. His research interests include wireless mobile communications, artificial intelligence security, network, and information security.